CHAPTER 2

Simple Neural Nets for Pattern Classification

2.1 GENERAL DISCUSSION

One of the simplest tasks that neural nets can be trained to perform is pattern classification. In pattern classification problems, each input vector (pattern) belongs, or does not belong, to a particular class or category. For a neural net approach, we assume we have a set of training patterns for which the correct classification is known. In the simplest case, we consider the question of membership in a single class. The output unit represents membership in the class with a response of 1; a response of -1 (or 0 if binary representation is used) indicates that the pattern is not a member of the class. For the single-layer nets described in this chapter, extension to the more general case in which each pattern may or may not belong to any of several classes is immediate. In such case, there is an output unit for each class. Pattern classification is one type of pattern recognition; the associative recall of patterns (discussed in Chapter 3) is another.

Pattern classification problems arise in many areas. In 1963, Donald Specht (a student of Widrow) used neural networks to detect heart abnormalities with EKG types of data as input (46 measurements). The output was "normal" or "abnormal," with an "on" response signifying normal [Specht, 1967; Caudill & Butler, 1990]. In the early 1960s, Minsky and Papert used neural nets to classify input patterns as convex or not convex and connected or not connected [Minsky & Papert, 1988]. There are many other examples of pattern classification problems

being solved by neural networks, both the simple nets described in this chapter, other early nets not discussed here, and multilayer nets (especially the backpropagation nets described in Chapter 6).

In this chapter, we shall discuss three methods of training a simple single-layer neural net for pattern classification: the Hebb rule, the perceptron learning rule, and the delta rule (used by Widrow in his ADALINE neural net). First, however, we discuss some issues that are common to all single-layer nets that perform pattern classification. We conclude the chapter with some comparisons of the nets discussed and an extension to a multilayer net, MADALINE.

Many real-world examples need more sophisticated architecture and training rules because the conditions for a single-layer net to be adequate (see Section 2.1.3) are not met. However, if the conditions are met approximately, the results may be sufficiently accurate. Also, insight can be gained from the more simple nets, since the meaning of the weights may be easier to interpret.

2.1.1 Architecture

The basic architecture of the simplest possible neural networks that perform pattern classification consists of a layer of input units (as many units as the patterns to be classified have components) and a single output unit. Most neural nets we shall consider in this chapter use the single-layer architecture shown in Figure 2.1. This allows classification of vectors, which are n-tuples, but considers membership in only one category.

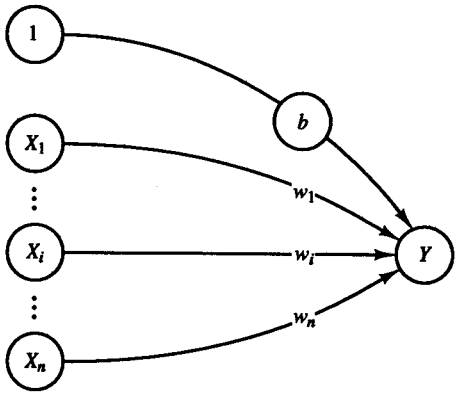

Figure 2.1 Single-layer net for pattern classification.

An example of a net that classifies the input into several categories is considered in Section 2.3.3. This net is a simple extension of the nets that perform

a single classification. The MADALINE net considered in Section 2.4.5 is a multilayer extension of the single-layer ADALINE net.

2.1.2 Biases and Thresholds

A bias acts exactly as a weight on a connection from a unit whose activation is always 1. Increasing the bias increases the net input to the unit. If a bias is included, the activation function is typically taken to be

$$f(\text{net}) = \begin{cases} 1 & \text{if net} \geq 0; \\ -1 & \text{if net} < 0; \end{cases}$$

where

$$\text{net} = b + \sum_i x_i w_i.$$

Some authors do not use a bias weight, but instead use a fixed threshold θ for the activation function. In that case,

$$f(\text{net}) = \begin{cases} 1 & \text{if net} \geq \theta; \\ -1 & \text{if net} < \theta; \end{cases}$$

where

$$\text{net} = \sum_i x_i w_i.$$

However, as the next example will demonstrate, this is essentially equivalent to the use of an adjustable bias.

Example 2.1 The role of a bias or a threshold

In this example and in the next section, we consider the separation of the input space into regions where the response of the net is positive and regions where the response is negative. To facilitate a graphical display of the relationships, we illustrate the ideas for an input with two components while the output is a scalar (i.e., it has only one component). The architecture of these examples is given in Figure 2.2.

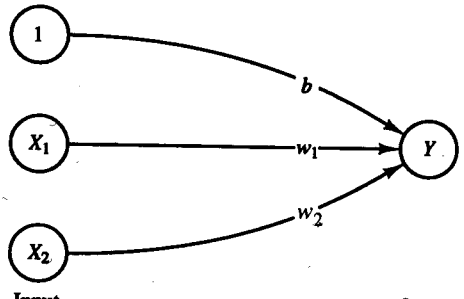

Figure 2.2 Single-layer neural network for logic functions.

The boundary between the values of x_1 and x_2 for which the net gives a positive response and the values for which it gives a negative response is the separating line

$$b + x_1 w_1 + x_2 w_2 = 0,$$

or (assuming that $w_2 \neq 0$),

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}.$$

The requirement for a positive response from the output unit is that the net input it receives, namely, $b + x_1 w_1 + x_2 w_2$, be greater than 0. During training, values of w_1, w_2, and b are determined so that the net will have the correct response for the training data.

If one thinks in terms of a threshold, the requirement for a positive response from the output unit is that the net input it receives, namely, $x_1 w_1 + x_2 w_2$, be greater than the threshold. This gives the equation of the line separating positive from negative output as

$$x_1 w_1 + x_2 w_2 = \theta,$$

or (assuming that $w_2 \neq 0$),

$$x_2 = -\frac{w_1}{w_2} x_1 + \frac{\theta}{w_2}.$$

During training, values of w_1 and w_2 are determined so that the net will have the correct response for the training data. In this case, the separating line cannot pass through the origin, but a line can be found that passes arbitrarily close to the origin.

The form of the separating line found by using an adjustable bias and the form obtained by using a fixed threshold illustrate that there is no advantage to including both a bias and a nonzero threshold for a neuron that uses the step function as its activation function. On the other hand, including neither a bias nor a threshold is equivalent to requiring the separating line (or plane or hyperplane for inputs with more components) to pass through the origin. This may or may not be appropriate for a particular problem.

As an illustration of a pseudopsychological analogy to the use of a bias, consider a simple (artificial) neural net in which the activation of the neuron corresponds to a person's action, "Go to the ball game." Each input signal corresponds to some factor influencing the decision to "go" or "not go" (other possible activities, the weather conditions, information about who is pitching, etc.). The weights on these input signals correspond to the importance the person places on each factor. (Of course, the weights may change with time, but methods for modifying them are not considered in this illustration.) A bias could represent a general inclination to "go" or "not go," based on past experiences. Thus, the bias would be modifiable, but the signal to it would not correspond to information about the specific game in question or activities competing for the person's time.

The threshold for this "decision neuron" indicates the total net input necessary to cause the person to "go," i.e., for the decision neuron to fire. The threshold would be different for different people; however, for the sake of this simple example, it should be thought of as a quantity that remains fixed for each individual. Since it is the relative values of the weights, rather than their actual magnitudes, that determine the response of the neuron, the model can cover all possibilities using either the fixed threshold or the adjustable bias.

2.1.3 Linear Separability

For each of the nets in this chapter, the intent is to train the net (i.e., adaptively determine its weights) so that it will respond with the desired classification when presented with an input pattern that it was trained on or when presented with one that is sufficiently similar to one of the training patterns. Before discussing the particular nets (which is to say, the particular styles of training), it is useful to discuss some issues common to all of the nets. For a particular output unit, the desired response is a "yes" if the input pattern is a member of its class and a "no" if it is not. A "yes" response is represented by an output signal of 1, a "no" by an output signal of -1 (for bipolar signals). Since we want one of two responses, the activation (or transfer or output) function is taken to be a step function. The value of the function is 1 if the net input is positive and -1 if the net input is negative. Since the net input to the output unit is

$$y\_in = b + \sum_i x_i w_i,$$

it is easy to see that the boundary between the region where $y\_in > 0$ and the region where $y\_in < 0$, which we call the *decision boundary*, is determined by the relation

$$b + \sum_i x_i w_i = 0.$$

Depending on the number of input units in the network, this equation represents a line, a plane, or a hyperplane.

If there are weights (and a bias) so that all of the training input vectors for which the correct response is $+1$ lie on one side of the decision boundary and all of the training input vectors for which the correct response is -1 lie on the other side of the decision boundary, we say that the problem is "linearly separable." Minsky and Papert [1988] showed that a single-layer net can learn only linearly separable problems. Furthermore, it is easy to extend this result to show that multilayer nets with linear activation functions are no more powerful than single-layer nets (since the composition of linear functions is linear).

It is convenient, if the input vectors are ordered pairs (or at most ordered triples), to graph the input training vectors and indicate the desired response by the appropriate symbol ("+" or "−"). The analysis also extends easily to nets

with more input units; however, the graphical display is not as convenient. The region where y is positive is separated from the region where it is negative by the line

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}.$$

These two regions are often called *decision regions* for the net. Notice in the following examples that there are many different lines that will serve to separate the input points that have different target values. However, for any particular line, there are also many choices of w_1, w_2, and b that give exactly the same line. The choice of the sign for b determines which side of the separating line corresponds to a $+1$ response and which side to a -1 response.

There are four different bipolar input patterns we can use to train a net with two input units. However, there are two possible responses for each input pattern, so there are 2^4 different functions that we might be able to train a very simple net to perform. Several of these functions are familiar from elementary logic, and we will use them for illustrations, for convenience. The first question we consider is, For this very simple net, do weights exist so that the net will have the desired output for each of the training input vectors?

Example 2.2 Response regions for the AND function

The AND function (for bipolar inputs and target) is defined as follows:

INPUT (x_1, x_2)	OUTPUT (t)
(1, 1)	+1
(1, −1)	−1
(−1, 1)	−1
(−1, −1)	−1

The desired responses can be illustrated as shown in Figure 2.3. One possible decision boundary for this function is shown in Figure 2.4.

An example of weights that would give the decision boundary illustrated in the figure, namely, the separating line

$$x_2 = -x_1 + 1,$$

is

$$\begin{aligned} b &= -1, \\ w_1 &= 1, \\ w_2 &= 1. \end{aligned}$$

The choice of sign for b is determined by the requirement that

$$b + x_1 w_1 + x_2 w_2 < 0$$

where $x_1 = 0$ and $x_2 = 0$. (Any point that is not on the decision boundary can be used to determine which side of the boundary is positive and which is negative; the origin is particularly convenient to use when it is not on the boundary.)

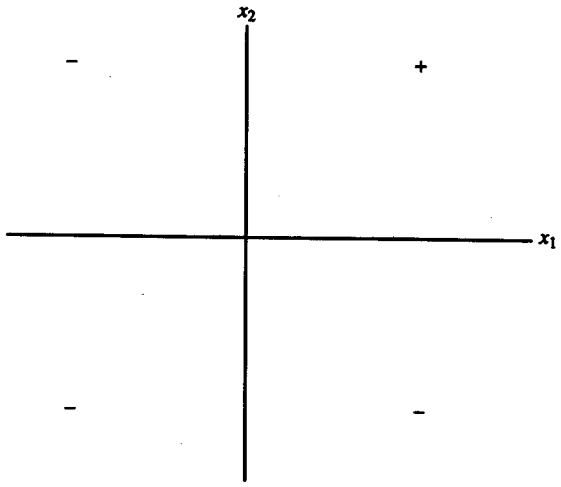

Figure 2.3 Desired response for the logic function AND (for bipolar inputs).

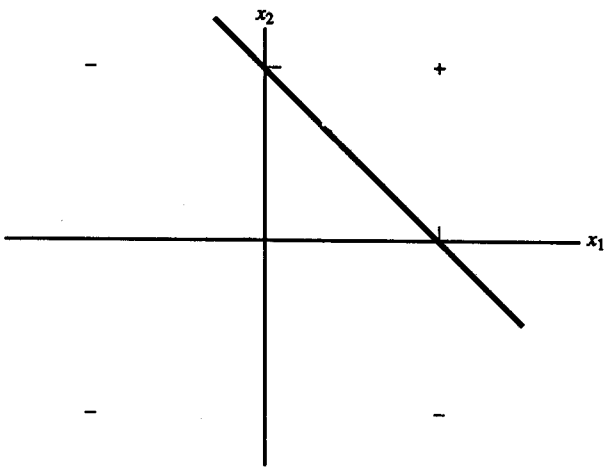

Figure 2.4 The logic function AND, showing a possible decision boundary.

Example 2.3 Response regions for the OR function

The logical OR function (for bipolar inputs and target) is defined as follows:

INPUT (x_1, x_2)	OUTPUT (t)
(1, 1)	+1
(1, −1)	+1
(−1, 1)	+1
(−1, −1)	−1

The weights must be chosen to provide a separating line, as illustrated in Figure 2.5. One example of suitable weights is

$$b = 1,$$

$$w_1 = 1,$$

$$w_2 = 1,$$

giving the separating line

$$x_2 = -x_1 - 1.$$

The choice of sign for b is determined by the requirement that

$$b + x_1 w_1 + x_2 w_2 > 0$$

where $x_1 = 0$ and $x_2 = 0$.

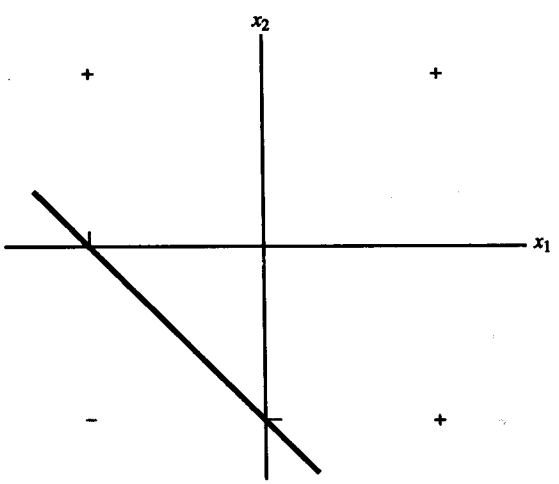

Figure 2.5 The logic function OR, showing a possible decision boundary.

The preceding two mappings (which can each be solved by a single-layer neural net) illustrate graphically the concept of linearly separable input. The input points to be classified positive can be separated from the input points to be classified negative by a straight line. The equations of the decision boundaries are not unique. We will return to these examples to illustrate each of the learning rules in this chapter.

Note that if a bias weight were not included in these examples, the decision boundary would be forced to go through the origin. In many cases (including Examples 2.2 and 2.3), that would change a problem that could be solved (i.e., learned, or one for which weights exist) into a problem that could not be solved.

Not all simple two-input, single-output mappings can be solved by a single-layer net (even with a bias included), as is illustrated in Example 2.4.

Example 2.4 Response regions for the XOR function

The desired response of this net is as follows:

INPUT (x_1, x_2)	OUTPUT (t)
(1, 1)	−1
(1, −1)	+1
(−1, 1)	+1
(−1, −1)	−1

It is easy to see that no single straight line can separate the points for which a positive response is desired from those for which a negative response is desired.

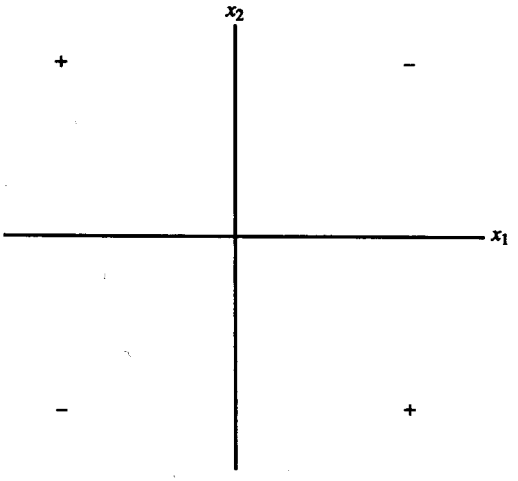

Figure 2.6 Desired response for the logic function XOR.

2.1.4 Data Representation

The previous examples show the use of a bipolar (values 1 and -1) representation of the training data, rather than the binary representation used for the McCulloch-Pitts neurons in Chapter 1. Many early neural network models used binary representation, although in most cases it can be modified to bipolar form. The form of the data may change the problem from one that can be solved by a simple neural net to one that cannot, as is illustrated in Examples 2.5–2.7 for the Hebb rule. Binary representation is also not as good as bipolar if we want the net to generalize (i.e., respond to input data similar, but not identical to, training data). Using bipolar input, missing data can be distinguished from mistaken data. Missing values can be represented by "0" and mistakes by reversing the input value from $+1$ to -1, or vice versa. We shall discuss some of the issues relating to the choice of binary versus bipolar representation further as they apply to particular neural nets. In general, bipolar representation is preferable.

The remainder of this chapter focuses on three methods of training single-layer neural nets that are useful for pattern classification: the Hebb rule, the perceptron learning rule, and the delta rule (or least mean squares). The Hebb rule, or correlational learning, is extremely simple but limited (even for linearly separable problems); the training algorithms for the perceptron and for ADALINE (adaptive linear neuron, trained by the delta rule) are closely related. Both are iterative techniques that are guaranteed to converge under suitable circumstances. The generalization of an ADALINE to a multilayer net (MADALINE) also will be examined.

2.2 HEBB NET

The earliest and simplest learning rule for a neural net is generally known as the Hebb rule. Hebb proposed that learning occurs by modification of the synapse strengths (weights) in a manner such that if two interconnected neurons are both "on" at the same time, then the weight between those neurons should be increased. The original statement only talks about neurons firing at the same time (and does not say anything about reinforcing neurons that do not fire at the same time). However, a stronger form of learning occurs if we also increase the weights if both neurons are "off" at the same time. We use this extended Hebb rule [McClelland & Rumelhart, 1988] because of its improved computational power and shall refer to it as the Hebb rule.

We shall refer to a single-layer (feedforward) neural net trained using the (extended) Hebb rule as a *Hebb net*. The Hebb rule is also used for training other specific nets that are discussed later. Since we are considering a single-layer net, one of the interconnected neurons will be an input unit and one an output unit (since no input units are connected to each other, nor are any output units in-

terconnected). If data are represented in bipolar form, it is easy to express the desired weight update as

$$w_i(\text{new}) = w_i(\text{old}) + x_i y.$$

If the data are binary, this formula does not distinguish between a training pair in which an input unit is "on" and the target value is "off" and a training pair in which both the input unit and the target value are "off." Examples 2.5 and 2.6 (in Section 2.2.2) illustrate the extreme limitations of the Hebb rule for binary data. Example 2.7 shows the improved performance achieved by using bipolar representation for both the input and target values.

2.2.1 Algorithm

Step 0. Initialize all weights:

$$w_i = 0 \qquad (i = 1 \text{ to } n).$$

Step 1. For each input training vector and target output pair, **s** : t, do steps 2–4.

Step 2. Set activations for input units:

$$x_i = s_i \qquad (i = 1 \text{ to } n).$$

Step 3. Set activation for output unit:

$$y = t.$$

Step 4. Adjust the weights for

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \qquad (i = 1 \text{ to } n).$$

Adjust the bias:

$$b(\text{new}) = b(\text{old}) + y.$$

Note that the bias is adjusted exactly like a weight from a "unit" whose output signal is always 1. The weight update can also be expressed in vector form as

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \mathbf{x}y.$$

This is often written in terms of the weight change, $\Delta\mathbf{w}$, as

$$\Delta\mathbf{w} = \mathbf{x}y$$

and

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \Delta\mathbf{w}.$$

There are several methods of implementing the Hebb rule for learning. The foregoing algorithm requires only one pass through the training set; other equivalent methods of finding the weights are described in Section 3.1.1, where the Hebb rule for pattern association (in which the target is a vector) is presented.

2.2.2 Application

Bias types of inputs are not explicitly used in the original formulation of Hebb learning. However, they are included in the examples in this section (shown as a third input component that is always 1) because without them, the problems discussed cannot be solved.

Logic functions

Example 2.5 A Hebb net for the AND function: binary inputs and targets

INPUT			TARGET
$(x_1$	x_2	1)	
(1	1	1)	1
(1	0	1)	0
(0	1	1)	0
(0	0	1)	0

For each training input: target, the weight change is the product of the input vector and the target value, i.e.,

$$\Delta w_1 = x_1 t, \qquad \Delta w_2 = x_2 t, \qquad \Delta b = t.$$

The new weights are the sum of the previous weights and the weight change. Only one iteration through the training vectors is required. The weight updates for the first input are as follows:

INPUT			TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)		$(\Delta w_1$	Δw_2	$\Delta b)$	$(w_1$	w_2	$b)$
							(0	0	0)
(1	1	1)	1	(1	1	1)	(1	1	1)

The separating line (see Section 2.1.3) becomes

$$x_2 = -x_1 - 1.$$

The graph, presented in Figure 2.7, shows that the response of the net will now be correct for the first input pattern. Presenting the second, third, and fourth training inputs shows that because the target value is 0, no learning occurs. Thus, using binary target values prevents the net from learning any pattern for which the target is "off":

INPUT			TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)		$(\Delta w_1$	Δw_2	$\Delta b)$	$(w_1$	w_2	$b)$
(1	0	1)	0	(0	0	0)	(1	1	1)
(0	1	1)	0	(0	0	0)	(1	1	1)
(0	0	1)	0	(0	0	0)	(1	1	1)

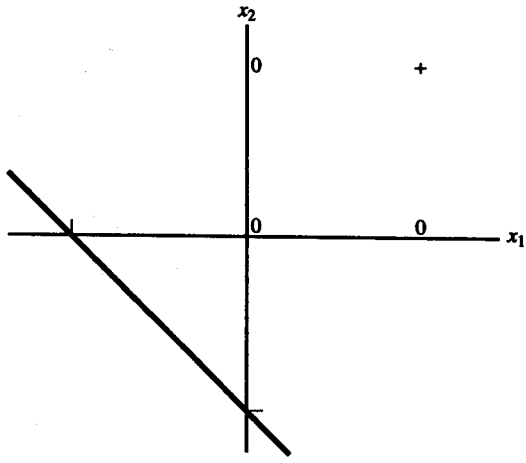

Figure 2.7 Decision boundary for binary AND function using Hebb rule after first training pair.

Example 2.6 A Hebb net for the AND function: binary inputs, bipolar targets

INPUT			TARGET
(x_1	x_2	1)	
(1	1	1)	1
(1	0	1)	−1
(0	1	1)	−1
(0	0	1)	−1

Presenting the first input, including a value of 1 for the third component, yields the following:

INPUT			TARGET	WEIGHT CHANGES			WEIGHTS		
(x_1	x_2	1)		(Δw_1	Δw_2	Δb)	(w_1	w_2	b)
							(0	0	0)
(1	1	1)	1	(1	1	1)	(1	1	1)

The separating line becomes

$$x_2 = -x_1 - 1.$$

Figure 2.8 shows that the response of the net will now be correct for the first input pattern.

Presenting the second, third, and fourth training patterns shows that learning continues for each of these patterns (since the target value is now −1, rather than 0, as in Example 2.5).

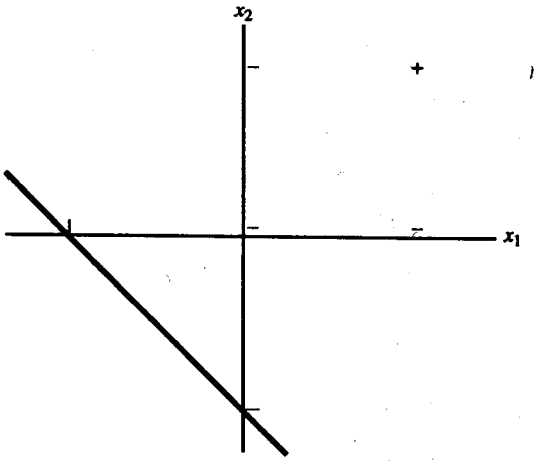

Figure 2.8 Decision boundary for AND function using Hebb rule after first training pair (binary inputs, bipolar targets).

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$		$(\Delta w_1 \quad \Delta w_2 \quad \Delta b)$	$(w_1 \quad w_2 \quad b)$
$(1 \quad 0 \quad 1)$	-1	$(-1 \quad 0 \quad -1)$	$(0 \quad 1 \quad 0)$
$(0 \quad 1 \quad 1)$	-1	$(0 \quad -1 \quad -1)$	$(0 \quad 0 \quad -1)$
$(0 \quad 0 \quad 1)$	-1	$(0 \quad 0 \quad -1)$	$(0 \quad 0 \quad -2)$

However, these weights do not provide the correct response for the first input pattern.

The choice of training patterns can play a significant role in determining which problems can be solved using the Hebb rule. The next example shows that the AND function can be solved if we modify its representation to express the inputs as well as the targets in bipolar form. Bipolar representation of the inputs and targets allows modification of a weight when the input unit and the target value are both "on" at the same time and when they are both "off" at the same time. The algorithm is the same as that just given, except that now all units will learn whenever there is an error in the output.

Example 2.7 A Hebb net for the AND function: bipolar inputs and targets

INPUT	TARGET
$(x_1 \quad x_2 \quad 1)$	
$(1 \quad 1 \quad 1)$	1
$(1 \quad -1 \quad 1)$	-1
$(-1 \quad 1 \quad 1)$	-1
$(-1 \quad -1 \quad 1)$	-1

Presenting the first input, including a value of 1 for the third component, yields the following:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$		$(\Delta w_1 \quad \Delta w_2 \quad \Delta b)$	$(w_1 \quad w_2 \quad b)$
			(0 0 0)
(1 1 1)	1	(1 1 1)	(1 1 1)

The separating line becomes

$$x_2 = -x_1 - 1.$$

The graph in Figure 2.9 shows that the response of the net will now be correct for the first input point (and also, by the way, for the input point $(-1, -1)$). Presenting the second input vector and target results in the following situation:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$		$(\Delta w_1 \quad \Delta w_2 \quad \Delta b)$	$(w_1 \quad w_2 \quad b)$
			(1 1 1)
(1 −1 1)	−1	(−1 1 −1)	(0 2 0)

The separating line becomes

$$x_2 = 0.$$

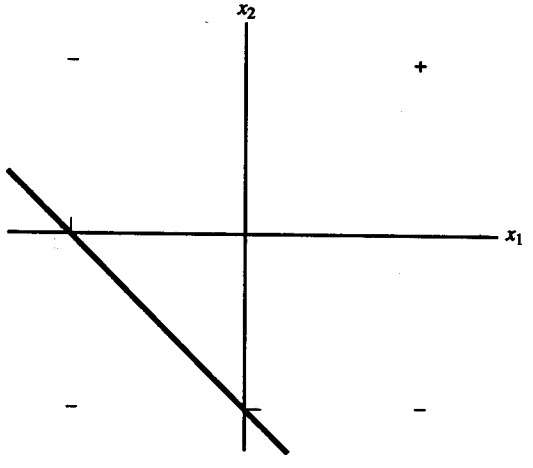

Figure 2.9 Decision boundary for the AND function using Hebb rule after first training pair (bipolar inputs and targets).

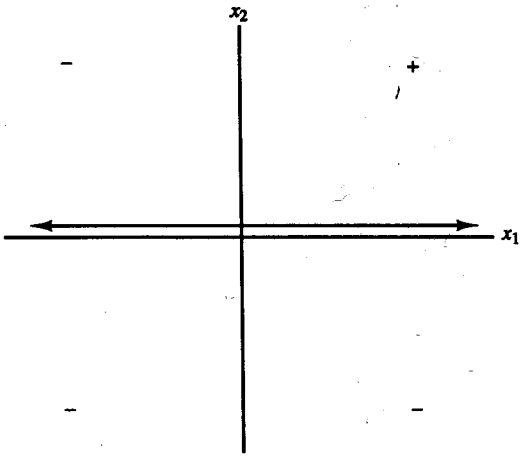

Figure 2.10 Decision boundary for bipolar AND function using Hebb rule after second training pattern (boundary is x_1-axis).

The graph in Figure 2.10 shows that the response of the net will now be correct for the first two input points, (1, 1) and (1, −1), and also, incidentally, for the input point (−1, −1). Presenting the third input vector and target yields the following:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$		$(\Delta w_1 \quad \Delta w_2 \quad \Delta b)$	$(w_1 \quad w_2 \quad b)$
			(0 2 0)
(−1 1 1)	−1	(1 −1 −1)	(1 1 −1)

The separating line becomes

$$x_2 = -x_1 + 1.$$

The graph in Figure 2.11 shows that the response of the net will now be correct for the first three input points (and also, by the way, for the input point (−1, −1)). Presenting the last point, we obtain the following:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$		$(\Delta w_1 \quad \Delta w_2 \quad \Delta b)$	$(w_1 \quad w_2 \quad b)$
			(1 1 −1)
(−1 −1 1)	−1	(1 1 −1)	(2 2 −2)

Even though the weights have changed, the separating line is still

$$x_2 = -x_1 + 1,$$

so the graph of the decision regions (the positive response and the negative response) remains as in Figure 2.11.

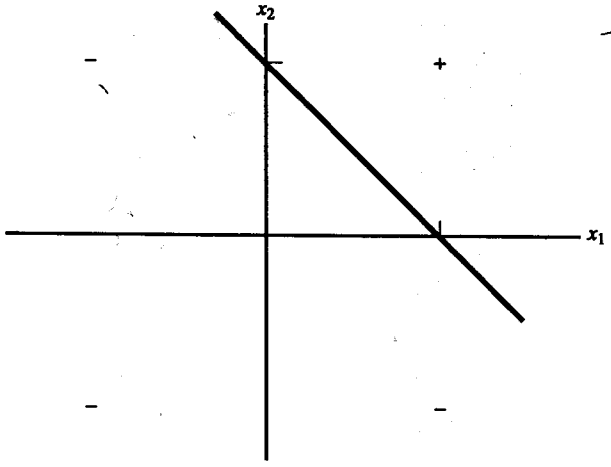

Figure 2.11 Decision boundary for bipolar AND function using Hebb rule after third training pattern.

Character recognition

Example 2.8 A Hebb net to classify two-dimensional input patterns (representing letters)

A simple example of using the Hebb rule for character recognition involves training the net to distinguish between the pattern "X" and the pattern "O". The patterns can be represented as

```
# . . . #        . # # # .
. # . # .        #       #
. . # . .   and  #       #
. # . # .        #       #
# . . . #        . # # # .
```

Pattern 1 Pattern 2

To treat this example as a pattern classification problem with one output class, we will designate that class "X" and take the pattern "O" to be an example of output that is not "X."

The first thing we need to do is to convert the patterns to input vectors. That is easy to do by assigning each # the value 1 and each "." the value −1. To convert from the two-dimensional pattern to an input vector, we simply concatenate the rows, i.e., the second row of the pattern comes after the first row, the third row follows, ect. Pattern 1 then becomes

1 −1 −1 −1 1, −1 1 −1 1 −1, −1 −1 1 −1 −1, −1 1 −1 1 −1,
1 −1 −1 −1 1,

and pattern 2 becomes

−1 1 1 1 −1, 1 −1 −1 −1 1, 1 −1 −1 −1 1, 1 −1 −1 −1 1, −1 1 1 1 −1,

where a comma denotes the termination of a line of the original matrix. For computer simulations, the program can be written so that the vector is read in from the two-dimensional format.

The correct response for the first pattern is "on," or +1, so the weights after presenting the first pattern are simply the input pattern. The bias weight after presenting this is +1. The correct response for the second pattern is "off," or −1, so the weight change when the second pattern is presented is

1 −1 −1 −1 1, −1 1 1 1 −1, −1 1 1 1 −1, −1 1 1 1 −1, 1 −1 −1 −1 1.

In addition, the weight change for the bias weight is −1.

Adding the weight change to the weights representing the first pattern gives the final weights:

2 −2 −2 −2 2, −2 2 0 2 −2, −2 0 2 0 −2, −2 2 0 2 −2, 2 −2 −2 −2 2.

The bias weight is 0.

Now, we compute the output of the net for each of the training patterns. The net input (for any input pattern) is the dot product of the input pattern with the weight vector. For the first training vector, the net input is 42, so the response is positive, as desired. For the second training pattern, the net input is −42, so the response is clearly negative, also as desired.

However, the net can also give reasonable responses to input patterns that are similar, but not identical, to the training patterns. There are two types of changes that can be made to one of the input patterns that will generate a new input pattern for which it is reasonable to expect a response. The first type of change is usually referred to as "mistakes in the data." In this case, one or more components of the input vector (corresponding to one or more pixels in the original pattern) have had their sign reversed, changing a 1 to a −1, or vice versa. The second type of change is called "missing data." In this situation, one or more of the components of the input vector have the value 0, rather than 1 or −1. In general, a net can handle more missing components than wrong components; in other words, with input data, "It's better not to guess."

Other simple examples

Example 2.9 Limitations of Hebb rule training for binary patterns

This example shows that the Hebb rule may fail, even if the problem is linearly separable (and even if 0 is not the target).

Consider the following input and target output pairs:

1	1	1	→	1
1	1	0	→	0
1	0	1	→	0
0	1	1	→	0

It is easy to see that the Hebb rule cannot learn any pattern for which the target is 0. So we must at least convert the targets to +1 and −1. Now consider

1	1	1	→	1
1	1	0	→	−1
1	0	1	→	−1
0	1	1	→	−1

Figure 2.12 shows that the problem is now solvable, i.e., the input points classified in one class (with target value +1) are linearly separable from those not in the class (with target value −1). The figure also shows that a nonzero bias will be necessary, since the separating plane does not pass through the origin. The plane pictured is $x_1 + x_2 + x_3 + (-2.5) = 0$, i.e., a weight vector of (1 1 1) and a bias of −2.5.

The weights (and bias) are found by taking the sum of the weight changes that occur at each stage of the algorithm. The weight change is simply the input pattern (augmented by the fourth component, the input to the bias weight, which is always 1) multiplied by the target value for the pattern. We obtain:

Weight change for first input pattern:	1	1	1	1
Weight change for second input pattern:	−1	−1	0	−1
Weight change for third input pattern:	−1	0	−1	−1
Weight change for fourth input pattern:	0	−1	−1	−1
Final weights (and bias)	−1	−1	−1	−2

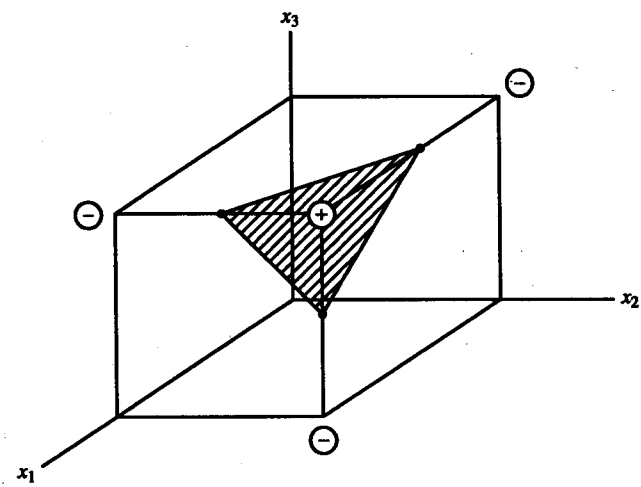

Figure 2.12 Linear separation of binary training inputs.

It is easy to see that these weights do not produce the correct output for the first pattern.

Example 2.10 Limitation of Hebb rule training for bipolar patterns

Examples 2.5, 2.6, and 2.7 show that even if the representation of the vectors does not change the problem from unsolvable to solvable, it can affect whether the Hebb rule works. In this example, we consider the same problem as in Example 2.9, but with the input points (and target classifications) in bipolar form. Accordingly, we have the following arrangement of values:

INPUT				TARGET	WEIGHT CHANGE				WEIGHT			
(x_1	x_2	x_3	1)		(Δw_1	Δw_2	Δw_3	Δb)	(w_1	w_2	w_3	b)
									(0	0	0	0)
(1	1	1	1)	1	(1	1	1	1)	(1	1	1	1)
(1	1	−1	1)	−1	(−1	−1	1	−1)	(0	0	2	0)
(1	−1	1	1)	−1	(−1	1	−1	−1)	(−1	1	1	−1)
(−1	1	1	1)	−1	(1	−1	−1	−1)	(0	0	0	−2)

Again, it is clear that the weights do not give the correct output for the first input pattern.

Figure 2.13 shows that the input points are linearly separable; one posssible plane, $x_1 + x_2 + x_3 + (-2) = 0$, to perform the separation is shown. This plane corresponds to a weight vector of (1 1 1) and a bias of −2.

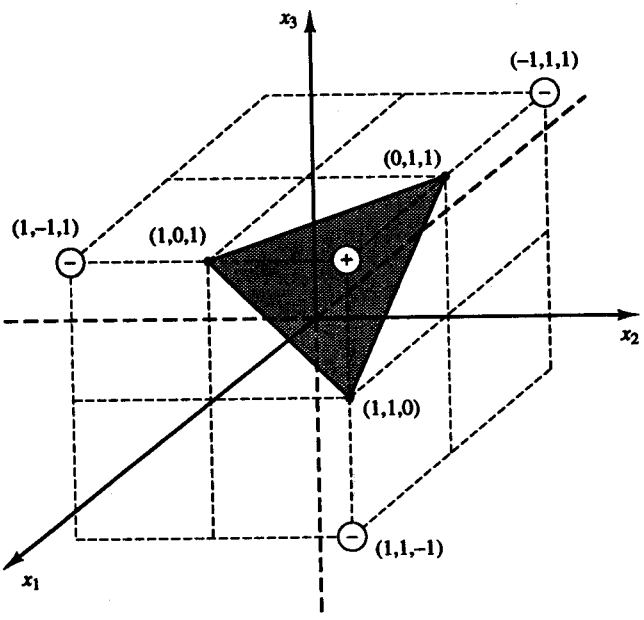

Figure 2.13 Linear separation of bipolar training inputs.

2.3 PERCEPTRON

Perceptrons had perhaps the most far-reaching impact of any of the early neural nets. The perceptron learning rule is a more powerful learning rule than the Hebb rule. Under suitable assumptions, its iterative learning procedure can be proved to converge to the correct weights, i.e., the weights that allow the net to produce the correct output value for each of the training input patterns. Not too surprisingly, one of the necessary assumptions is that such weights exist.

A number of different types of perceptrons are described in Rosenblatt (1962) and in Minsky and Papert (1969, 1988). Although some perceptrons were self-organizing, most were trained. Typically, the original perceptrons had three layers of neurons—sensory units, associator units, and a response unit—forming an approximate model of a retina. One particular simple perceptron [Block, 1962] used binary activations for the sensory and associator units and an activation of $+1$, 0, or -1 for the response unit. The sensory units were connected to the associator units by connections with fixed weights having values of $+1$, 0, or -1, assigned at random.

The activation function for each associator unit was the binary step function with an arbitrary, but fixed, threshold. Thus, the signal sent from the associator units to the output unit was a binary (0 or 1) signal. The output of the perceptron is $y = f(y\_in)$, where the activation function is

$$f(y\_in) = \begin{cases} 1 & \text{if } y\_in > \theta \\ 0 & \text{if } -\theta \leq y\_in \leq \theta \\ -1 & \text{if } y\_in < -\theta \end{cases}$$

The weights from the associator units to the response (or output) unit were adjusted by the perceptron learning rule. For each training input, the net would calculate the response of the output unit. Then the net would determine whether an error occurred for this pattern (by comparing the calculated output with the target value). The net did not distinguish between an error in which the calculated output was zero and the target -1, as opposed to an error in which the calculated output was $+1$ and the target -1. In either of these cases, the sign of the error denotes that the weights should be changed in the direction indicated by the target value. However, only the weights on the connections from units that sent a non-zero signal to the output unit would be adjusted (since only these signals contributed to the error). If an error occurred for a particular training input pattern, the weights would be changed according to the formula

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i,$$

where the target value t is $+1$ or -1 and α is the learning rate. If an error did not occur, the weights would not be changed.

Training would continue until no error occurred. The perceptron learning rule convergence theorem states that if weights exist to allow the net to respond correctly to all training patterns, then the rule's procedure for adjusting the weights will find values such that the net does respond correctly to all training patterns (i.e., the net solves the problem or learns the classification). Moreover, the net will find these weights in a finite number of training steps. We will consider a proof of this theorem in Section 2.3.4, since it helps clarify which aspects of the many variations on perceptron learning are significant.

2.3.1 Architecture

Simple perceptron for pattern classification

The output from the associator units in the original simple perceptron was a binary vector; that vector is treated as the input signal to the output unit in the sections that follow. As the proof of the perceptron learning rule convergence theorem given in Section 2.3.4 illustrates, the assumption of a binary input is not necessary. Since only the weights from the associator units to the output unit could be adjusted, we limit our consideration to the single-layer portion of the net, shown in Figure 2.14. Thus, the associator units function like input units, and the architecture is as given in the figure.

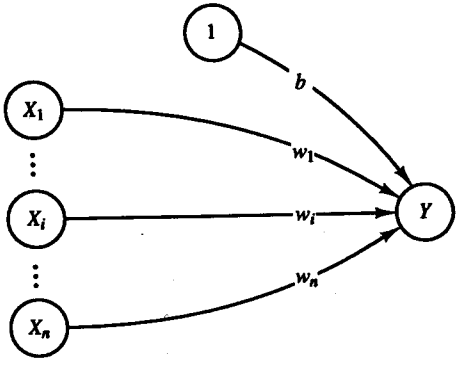

Figure 2.14 Perceptron to perform single classification.

The goal of the net is to classify each input pattern as belonging, or not belonging, to a particular class. Belonging is signified by the output unit giving a response of $+1$; not belonging is indicated by a response of -1. The net is trained to perform this classification by the iterative technique described earlier and given in the algorithm that follows.

2.3.2 Algorithm

The algorithm given here is suitable for either binary or bipolar input vectors (n-tuples), with a bipolar target, fixed θ, and adjustable bias. The threshold θ does not play the same role as in the step function illustrated in Section 2.1.2; thus, both a bias and a threshold are needed. The role of the threshold is discussed following the presentation of the algorithm. The algorithm is not particularly sensitive to the initial values of the weights or the value of the learning rate.

Step 0. Initialize weights and bias.
(For simplicity, set weights and bias to zero.)
Set learning rate α ($0 < \alpha \leq 1$).
(For simplicity, α can be set to 1.)

Step 1. While stopping condition is false, do Steps 2–6.

Step 2. For each training pair s:t, do Steps 3–5.

Step 3. Set activations of input units:

$$x_i = s_i.$$

Step 4. Compute response of output unit:

$$y\_in = b + \sum_i x_i w_i;$$

$$y = \begin{cases} 1 & \text{if } y\_in > \theta \\ 0 & \text{if } -\theta \leq y\_in \leq \theta \\ -1 & \text{if } y\_in < -\theta \end{cases}$$

Step 5. Update weights and bias if an error occurred for this pattern.

If $y \neq t$,

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i,$$

$$b(\text{new}) = b(\text{old}) + \alpha t.$$

else

$$w_i(\text{new}) = w_i(\text{old}),$$

$$b(\text{new}) = b(\text{old}).$$

Step 6. Test stopping condition:
If no weights changed in Step 2, stop; else, continue.

Note that only weights connecting active input units ($x_i \neq 0$) are updated. Also, weights are updated only for patterns that do not produce the correct value of y. This means that as more training patterns produce the correct response, less learning occurs. This is in contrast to the training of the ADALINE units described in Section 2.4, in which learning is based on the difference between y_in and t.

The threshold on the activation function for the response unit is a fixed, non-negative value θ. The form of the activation function for the output unit (response unit) is such that there is an "undecided" band (of fixed width determined by θ) separating the region of positive response from that of negative response. Thus, the previous analysis of the interchangeability of bias and threshold does not apply, because changing θ would change the width of the band, not just the position.

Note that instead of one separating line, we have a line separating the region of positive response from the region of zero response, namely, the line bounding the inequality

$$w_1x_1 + w_2x_2 + b > \theta,$$

and a line separating the region of zero response from the region of negative response, namely, the line bounding the inequality

$$w_1x_1 + w_2x_2 + b < -\theta.$$

2.3.3 Application

Logic functions

Example 2.11 A Perceptron for the AND function: binary inputs, bipolar targets

Let us consider again the AND function with binary input and bipolar target, now using the perceptron learning rule. The training data are as given in Example 2.6 for the Hebb rule. An adjustable bias is included, since it is necessary if a single-layer net is to be able to solve this problem. For simplicity, we take $\alpha = 1$ and set the initial weights and bias to 0, as indicated. However, to illustrate the role of the threshold, we take $\theta = .2$.

The weight change is $\Delta \mathbf{w} = t(x_1, x_2, 1)$ if an error has occurred and zero otherwise. Presenting the first input, we have:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	$1)$							$(w_1$	w_2	$b)$
									(0	0	0)
(1	1	1)	0	0	1	(1	1	1)	(1	1	1)

The separating lines become

$$x_1 + x_2 + 1 = .2$$

and

$$x_1 + x_2 + 1 = -.2.$$

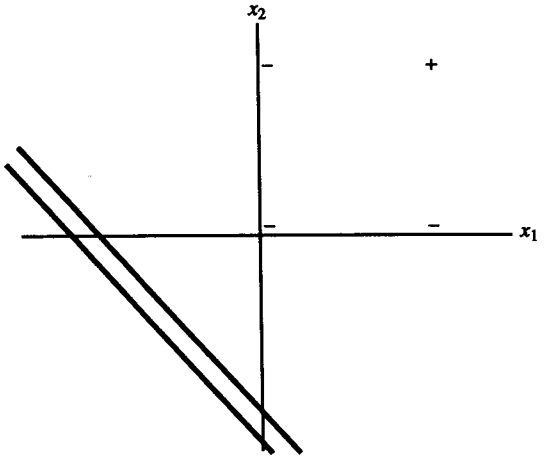

Figure 2.15 Decision boundary for logic function AND after first training input.

The graph in Figure 2.15 shows that the response of the net will now be correct for the first input pattern.

Presenting the second input yields the following:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(1	1	1)
(1	0	1)	2	1	−1	(−1	0	−1)	(0	①	0)

The separating lines become

$$x_2 = .2$$

and

$$x_2 = -.2$$

The graph in Figure 2.16 shows that the response of the net will now (still) be correct for the first input point.

For the third input, we have:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(0	1	0)
(0	1	1)	1	1	−1	(0	−1	−1)	(0	0	−1)

Since the components of the input patterns are nonnegative and the components of the weight vector are nonpositive, the response of the net will be negative (or zero).

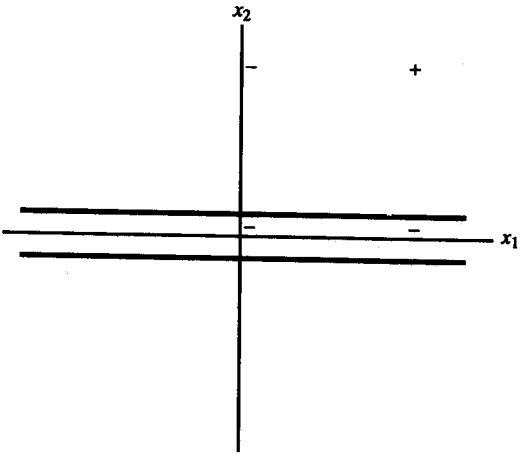

Figure 2.16 Decision boundary after second training input.

To complete the first epoch of training, we present the fourth training pattern:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(0	0	$-1)$
(0	0	1)	-1	-1	-1	(0	0	$-1)$	(0	0	$-1)$

The response for all of the input patterns is negative for the weights derived; but since the response for input pattern (1, 1) is not correct, we are not finished.

The second epoch of training yields the following weight updates for the first input:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(0	0	$-1)$
(1	1	1)	-1	-1	1	(1	1	1)	(1	1	0)

The separating lines become

$$x_1 + x_2 = .2$$

and

$$x_1 + x_2 = -.2.$$

The graph in Figure 2.17 shows that the response of the net will now be correct for (1, 1).

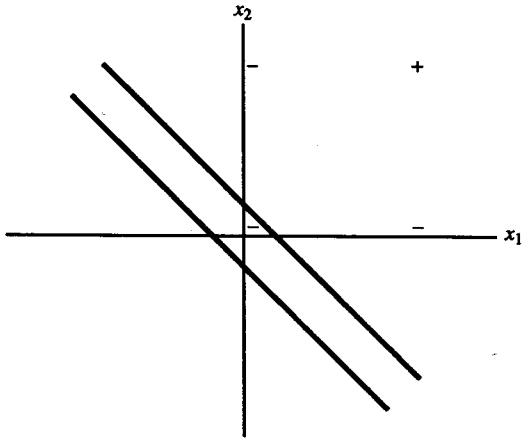

Figure 2.17 Boundary after first training input of second epoch.

For the second input in the second epoch, we have:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(1	1	0)
(1	0	1)	1	1	−1	(−1	0	−1)	(0	1	−1)

The separating lines become

$$x_2 - 1 = .2$$

and

$$x_2 - 1 = -.2.$$

The graph in Figure 2.18 shows that the response of the net will now be correct (negative) for the input points (1, 0) and (0, 0); the response for input points (0, 1) and (1, 1) will be 0, since the net input would be 0, which is between −.2 and .2 ($\theta = .2$).

In the second epoch, the third input yields:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(0	1	−1)
(0	1	1)	0	0	−1	(0	−1	−1)	(0	0	−2)

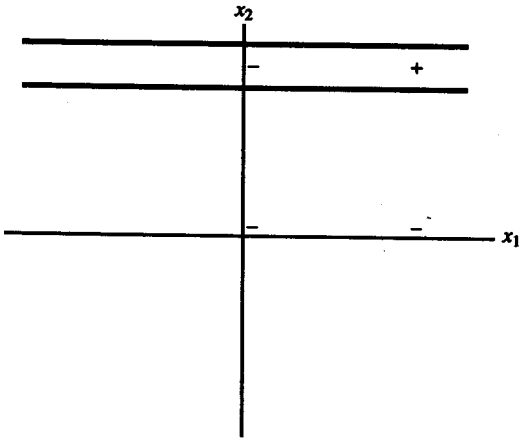

Figure 2.18 Boundary after second input of second epoch.

Again, the response will be negative for all of the input.

To complete the second epoch of training, we present the fourth training pattern:

INPUT			NET	OUT	TARGET	WEIGHTS CHANGE			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(0	0	−2)
(0	0	1)	−2	−1	−1	(0	0	0)	(0	0	−2)

The results for the third epoch are:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	1)							$(w_1$	w_2	$b)$
									(0	0	−2)
(1	1	1)	−2	−1	1	(1	1	1)	(1	1	−1)
(1	0	1)	0	0	−1	(−1	0	−1)	(0	1	−2)
(0	1	1)	−1	−1	1	(0	0	0)	(0	1	−2)
(0	0	1)	−2	−1	−1	(0	0	0)	(0	1	−2)

The results for the fourth epoch are:

(1	1	1)	−1	−1	1	(1	1	1)	(1	2	−1)
(1	0	1)	0	0	−1	(−1	0	−1)	(0	2	−2)
(0	1	1)	0	0	−1	(0	−1	−1)	(0	1	−3)
(0	0	1)	−3	−1	−1	(0	0	0)	(0	1	−3)

For the fifth epoch, we have

(1	1	1)	−2	−1	1	(1	1	1)	(1	2	−2)
(1	0	1)	−1	−1	−1	(0	0	0)	(1	2	−2)
(0	1	1)	0	0	−1	(0	−1	−1)	(1	1	−3)
(0	0	1)	−3	−1	−1	(0	0	0)	(1	1	−3)

and for the sixth epoch,

(1	1	1)	−1	−1	1	(1	1	1)	(2	2	−2)
(1	0	1)	0	0	−1	(−1	0	−1)	(1	2	−3)
(0	1	1)	−1	−1	−1	(0	0	0)	(1	2	−3)
(0	0	1)	−3	−1	−1	(0	0	0)	(1	2	−3)

The results for the seventh epoch are:

(1	1	1)	0	0	1	(1	1	1)	(2	3	−2)
(1	0	1)	0	0	−1	(−1	0	−1)	(1	3	−3)
(0	1	1)	0	0	−1	(0	−1	−1)	(1	2	−4)
(0	0	1)	−4	−1	−1	(0	0	0)	(1	2	−4)

The eighth epoch yields

(1	1	1)	−1	−1	1	(1	1	1)	(2	3	−3)
(1	0	1)	−1	−1	−1	(0	0	0)	(2	3	−3)
(0	1	1)	0	0	−1	(0	−1	−1)	(2	2	−4)
(0	0	1)	−4	−1	−1	(0	0	0)	(2	2	−4)

and the ninth

(1	1	1)	0	0	1	(1	1	1)	(3	3	−3)
(1	0	1)	0	0	−1	(−1	0	−1)	(2	3	−4)
(0	1	1)	−1	−1	−1	(0	0	0)	(2	3	−4)
(0	0	1)	−4	−1	−1	(0	0	0)	(2	3	−4)

Finally, the results for the tenth epoch are:

(1	1	1)	1	1	1	(0	0	0)	(2	3	−4)
(1	0	1)	−2	−1	−1	(0	0	0)	(2	3	−4)
(0	1	1)	−1	−1	−1	(0	0	0)	(2	3	−4)
(0	0	1)	−4	−1	−1	(0	0	0)	(2	3	−4)

Thus, the positive response is given by all points such that

$$2x_1 + 3x_2 - 4 > .2,$$

with boundary line

$$x_2 = -\frac{2}{3}x_1 + \frac{7}{5},$$

and the negative response is given by all points such that

$$2x_1 + 3x_2 - 4 < -.2,$$

with boundary line

$$x_2 = -\frac{2}{3}x_1 + \frac{19}{15}$$

(see Figure 2.19.)

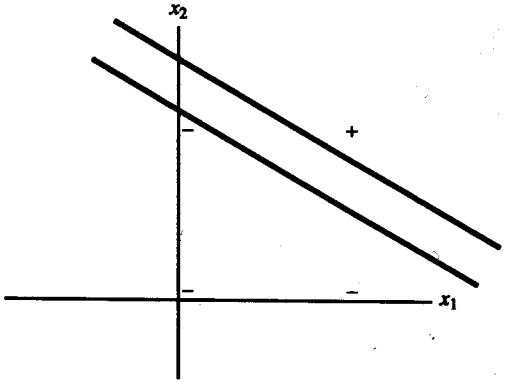

Figure 2.19 Final decision boundaries for AND function in perceptron learning.

Since the proof of the perceptron learning rule convergence theorem (Section 2.3.4) shows that binary input is not required, and in previous examples bipolar input was often preferable, we consider again the previous example, but with bipolar inputs, an adjustable bias, and $\theta = 0$. This variation provides the most direct comparison with Widrow-Hoff learning (an ADALINE net), which we consider in the next section. Note that it is not necessary to modify the training set so that all patterns are mapped to $+1$ (as is done in the proof of the perceptron learning rule convergence theorem); the weight adjustment is $t\mathbf{x}$ whenever the response of the net to input vector $\mathbf{x}$ is incorrect. The target value is still bipolar.

Example 2.12 A Perceptron for the AND function: bipolar inputs and targets

The training process for bipolar input, $\alpha = 1$, and threshold and initial weights = 0 is:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
(x_1	x_2	1)							(w_1	w_2	b)
									(0	0	0)
(1	1	1)	0	0	1	(1	1	1)	(1	1	1)
(1	−1	1)	1	1	−1	(−1	1	−1)	(0	2	0)
(−1	1	1)	2	1	−1	(1	−1	−1)	(1	1	−1)
(−1	−1	1)	−3	−1	−1	(0	0	0)	(1	1	−1)

In the second epoch of training, we have:

(1	1	1)	1	1	1	(0	0	0)	(1	1	−1)
(1	−1	1)	−1	−1	−1	(0	0	0)	(1	1	−1)
(−1	1	1)	−1	−1	−1	(0	0	0)	(1	1	−1)
(−1	−1	1)	−3	−1	−1	(0	0	0)	(1	1	−1)

Since all the Δw's are 0 in epoch 2, the system was fully trained after the first epoch.

It seems intuitively obvious that a procedure that could continue to learn to improve its weights even after the classifications are all correct would be better than a learning rule in which weight updates cease as soon as all training patterns are classified correctly. However, the foregoing example shows that the change from binary to bipolar representation improves the results rather spectacularly.

We next show that the perceptron with $\alpha = 1$ and $\theta = .1$ can solve the problem the Hebb rule could not.

Other simple examples

Example 2.13 Perceptron training is more powerful than Hebb rule training

The mapping of interest maps the first three components of the input vector onto a target value that is 1 if there are no zero inputs and that is −1 if there is one zero input. (If there are two or three zeros in the input, we do not specify the target value.) This is a portion of the parity problem for three inputs. The fourth component of the input vector is the input to the bias weight and is therefore always 1. The weight change vector is left blank if no error has occurred for a particular pattern. The learning rate is $\alpha = 1$, and the threshold $\theta = .1$. We show the following selected epochs:

INPUT	NET	OUT	TARGET	WEIGHT CHANGE	WEIGHTS
x_1 x_2 x_3 1					(w_1 w_2 w_3 b)
					(0 0 0 0)
Epoch 1:					
(1 1 1 1)	0	0	1	(1 1 1 1)	(1 1 1 1)
(1 1 0 1)	3	1	−1	(−1 −1 0 −1)	(0 0 1 0)
(1 0 1 1)	1	1	−1	(−1 0 −1 −1)	(−1 0 0 −1)
(0 1 1 1)	−1	−1	−1	( )	(−1 0 0 −1)
Epoch 2:					
(1 1 1 1)	−2	−1	1	(1 1 1 1)	(0 1 1 0)
(1 1 0 1)	1	1	−1	(−1 −1 0 −1)	(−1 0 1 −1)
(1 0 1 1)	−1	−1	−1	( )	(−1 0 1 −1)
(0 1 1 1)	0	0	−1	(0 −1 −1 −1)	(−1 −1 0 −2)
Epoch 3:					
(1 1 1 1)	−4	−1	1	(1 1 1 1)	(0 0 1 −1)
(1 1 0 1)	−1	−1	−1	( )	(0 0 1 −1)
(1 0 1 1)	0	0	−1	(−1 0 −1 −1)	(−1 0 0 −2)
(0 1 1 1)	−2	−1	−1	( )	(−1 0 0 −2)
Epoch 4:					
(1 1 1 1)	−3	−1	1	(1 1 1 1)	(0 1 1 −1)
(1 1 0 1)	0	0	−1	(−1 −1 0 −1)	(−1 0 1 −2)
(1 0 1 1)	−2	−1	−1	( )	(−1 0 1 −2)
(0 1 1 1)	−1	−1	−1	( )	(−1 0 1 −2)
Epoch 5:					
(1 1 1 1)	−2	−1	1	(1 1 1 1)	(0 1 2 −1)
(1 1 0 1)	0	0	−1	(−1 −1 0 −1)	(−1 0 2 −2)
(1 0 1 1)	−1	−1	−1	( )	(−1 0 2 −2)
(0 1 1 1)	0	0	−1	(0 −1 −1 −1)	(−1 −1 1 −3)
Epoch 10:					
(1 1 1 1)	−3	−1	1	(1 1 1 1)	(1 1 2 −3)
(1 1 0 1)	−1	−1	−1	( )	(1 1 2 −3)
(1 0 1 1)	0	0	−1	(−1 0 −1 −1)	(0 1 1 −4)
(0 1 1 1)	−2	−1	−1	( )	(0 1 1 −4)

Epoch 15:

(1	1	1	1)	−1	−1	1	(1	1	1	1)	(1	3	3	−4)
(1	1	0	1)	0	0	−1	(−1	−1	0	−1)	(0	2	3	−5)
(1	0	1	1)	−2	−1	−1	(			)	(0	2	3	−5)
(0	1	1	1)	0	0	−1	(0	−1	−1	−1)	(0	1	2	−6)

Epoch 20:

(1	1	1	1)	−2	−1	1	(1	1	1	1)	(2	2	4	−6)
(1	1	0	1)	−2	−1	−1	(			)	(2	2	4	−6)
(1	0	1	1)	0	0	−1	(−1	0	−1	−1)	(1	2	3	−7)
(0	1	1	1)	−2	−1	−1	(			)	(1	2	3	−7)

Epoch 25:

(1	1	1	1)	0	0	1	(1	1	1	1)	(3	4	4	−7)
(1	1	0	1)	0	0	−1	(−1	−1	0	−1)	(2	3	4	−8)
(1	0	1	1)	−2	−1	−1	(			)	(2	3	4	−8)
(0	1	1	1)	−1	−1	−1	(			)	(2	3	4	−8)

Epoch 26:

(1	1	1	1)	1	1	1	(			)	(2	3	4	−8)
(1	1	0	1)	−3	−1	−1	(			)	(2	3	4	−8)
(1	0	1	1)	−2	−1	−1	(			)	(2	3	4	−8)
(0	1	1	1)	−1	−1	−1	(			)	(2	3	4	−8)

Character recognition

Application Procedure

Step 0. Apply training algorithm to set the weights.

Step 1. For each input vector **x** to be classified, do Steps 2–3.

Step 2. Set activations of input units.

Step 3. Compute response of output unit:

$$y\_in = \sum_i x_i w_i;$$

$$y = \begin{cases} 1 & \text{if } y\_in > \theta \\ 0 & \text{if } -\theta \leq y\_in \leq \theta \\ -1 & \text{if } y\_in < -\theta \end{cases}$$

Example 2.14 A Perceptron to classify letters from different fonts: one output class

As the first example of using the perceptron for character recognition, consider the 21 input patterns in Figure 2.20 as examples of *A* or not-*A*. In other words, we train the perceptron to classify each of these vectors as belonging, or not belonging, to

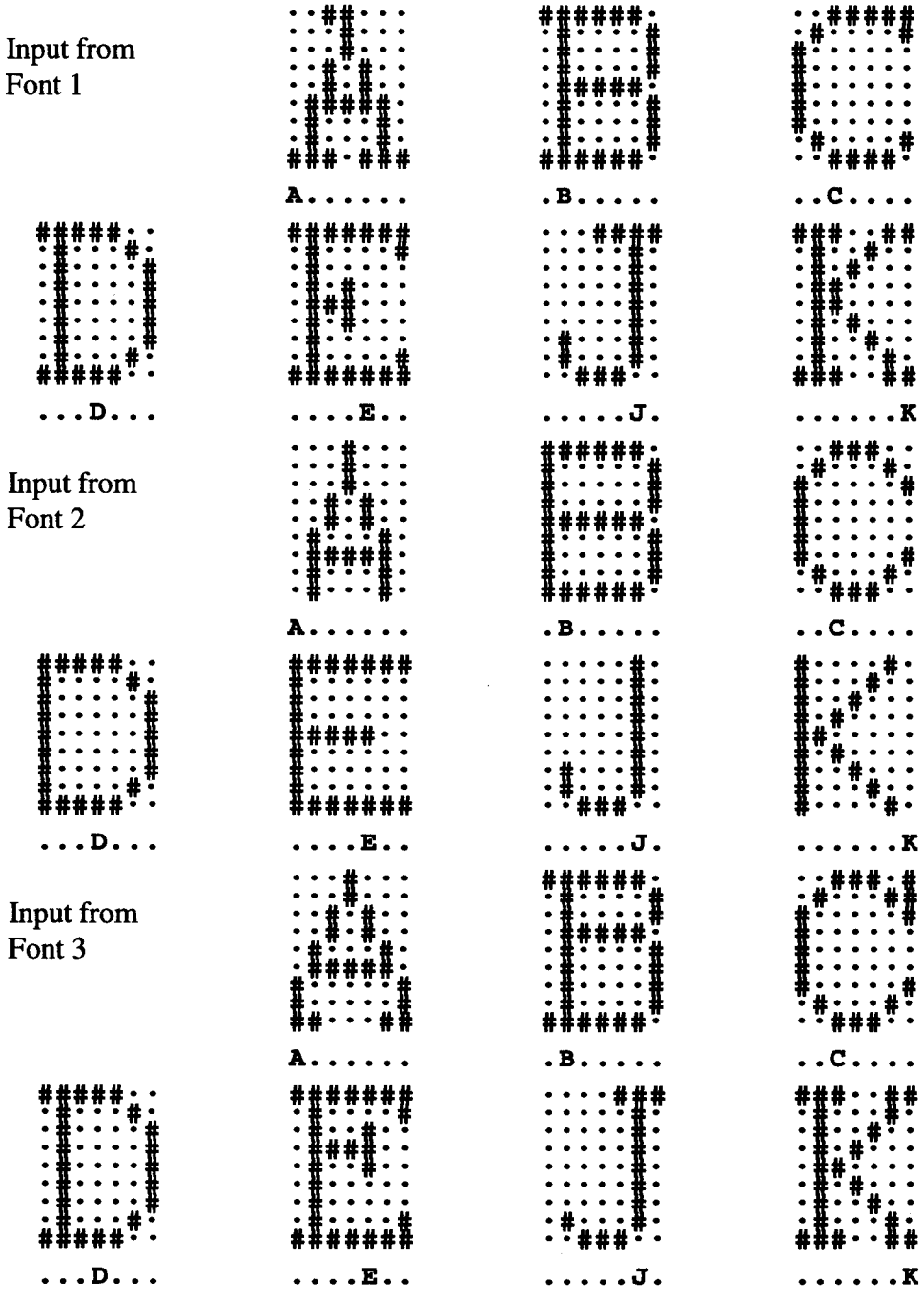

Figure 2.20 Training input and target output patterns.

the class A. In that case, the target value for each pattern is either 1 or -1; only the first component of the target vector shown is applicable. The net is as shown in Figure 2.14, and $n = 63$. There are three examples of A and 18 examples of not-A in Figure 2.20.

We could, of course, use the same vectors as examples of B or not-B and train the net in a similar manner. Note, however, that because we are using a single-layer net, the weights for the output unit signifying A do not have any interaction with the weights for the output unit signifying B. Therefore, we can solve these two problems at the same time, by allowing a column of weights for each output unit. Our net would have 63 input units and 2 output units. The first output unit would correspond to "A or not-A", the second unit to "B or not-B." Continuing this idea, we can identify 7 output units, one for each of the 7 categories into which we wish to classify our input.

Ideally, when an unknown character is presented to the net, the net's output consists of a single "yes" and six "nos." In practice, that may not happen, but the net may produce several guesses that can be resolved by other methods, such as considering the strengths of the activations of the various output units prior to setting the threshold or examining the context in which the ill-classified character occurs.

Example 2.15 A Perceptron to classify letters from different fonts: several output classes

The perceptron shown in Figure 2.14 can be extended easily to the case where the input vectors belong to one (or more) of several categories. In this type of application, there is an output unit representing each of the categories to which the input vectors may belong. The architecture of such a net is shown in Figure 2.21.

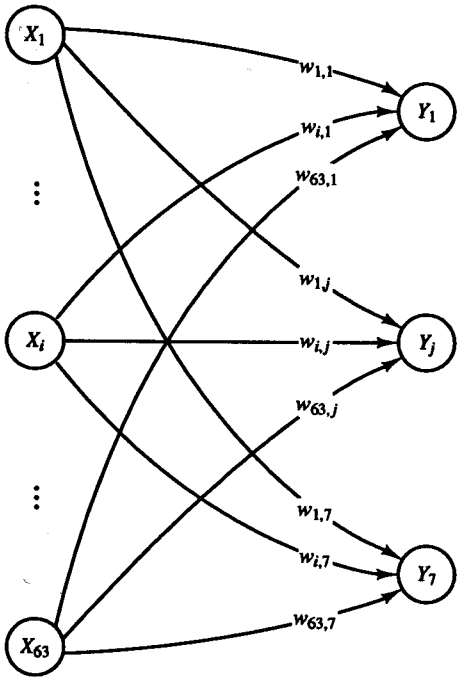

Figure 2.21 Perceptron to classify input into seven categories.

For this example, each input vector is a 63-tuple representing a letter expressed as a pattern on a 7 × 9 grid of pixels. The training patterns are illustrated in Figure 2.20. There are seven categories to which each input vector may belong, so there are seven components to the output vector, each representing a letter: A, B, C, D, E, K, or J. For ease of reading, we show the target output pattern indicating that the input was an "A" as (A · · · · · · ·), a "B" (· B · · · · · ·), etc.

The training input patterns and target responses must be converted to an appropriate form for the neural net to process. A bipolar representation has better computational characteristics than does a binary representation. The input patterns may be converted to bipolar vectors as described in Example 2.8; the target output pattern (A · · · · · · ·) becomes the bipolar vector (1, −1, −1, −1, −1, −1, −1) and the target pattern (· B · · · · · ·) is represented by the bipolar vector (−1, 1, −1, −1, −1, −1, −1).

A modified training algorithm for several output categories (threshold = 0, learning rate = 1, bipolar training pairs) is as follows:

Step 0. Initialize weights and biases
(0 or small random values).

Step 1. While stopping condition is false, do Steps 1–6.

Step 2. For each bipolar training pair **s : t**, do Steps 3–5.

Step 3. Set activation of each input unit, $i = 1, \ldots, n$:

$$x_i = s_i.$$

Step 4. Compute activation of each output unit, $j = 1, \ldots, m$:

$$y\_in_j = b_j + \sum_i x_i w_{ij}.$$

$$y_j = \begin{cases} 1 & \text{if } y\_in_j > \theta \\ 0 & \text{if } -\theta \le y\_in_j \le \theta \\ -1 & \text{if } y\_in_j < -\theta \end{cases}$$

Step 5. Update biases and weights, $j = 1, \ldots, m$; $i = 1, \ldots, n$:
If $t_j \neq y_j$, then

$$b_j(\text{new}) = b_j(\text{old}) + t_j;$$

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + t_j x_i.$$

Else, biases and weights remain unchanged.

Step 6. Test for stopping condition:
If no weight changes occurred in Step 2, stop; otherwise, continue.

After training, the net correctly classifies each of the training vectors.

The performance of the net shown in Figure 2.21 in classifying input vectors that are similar to the training vectors is shown in Figure 2.22. Each of the input

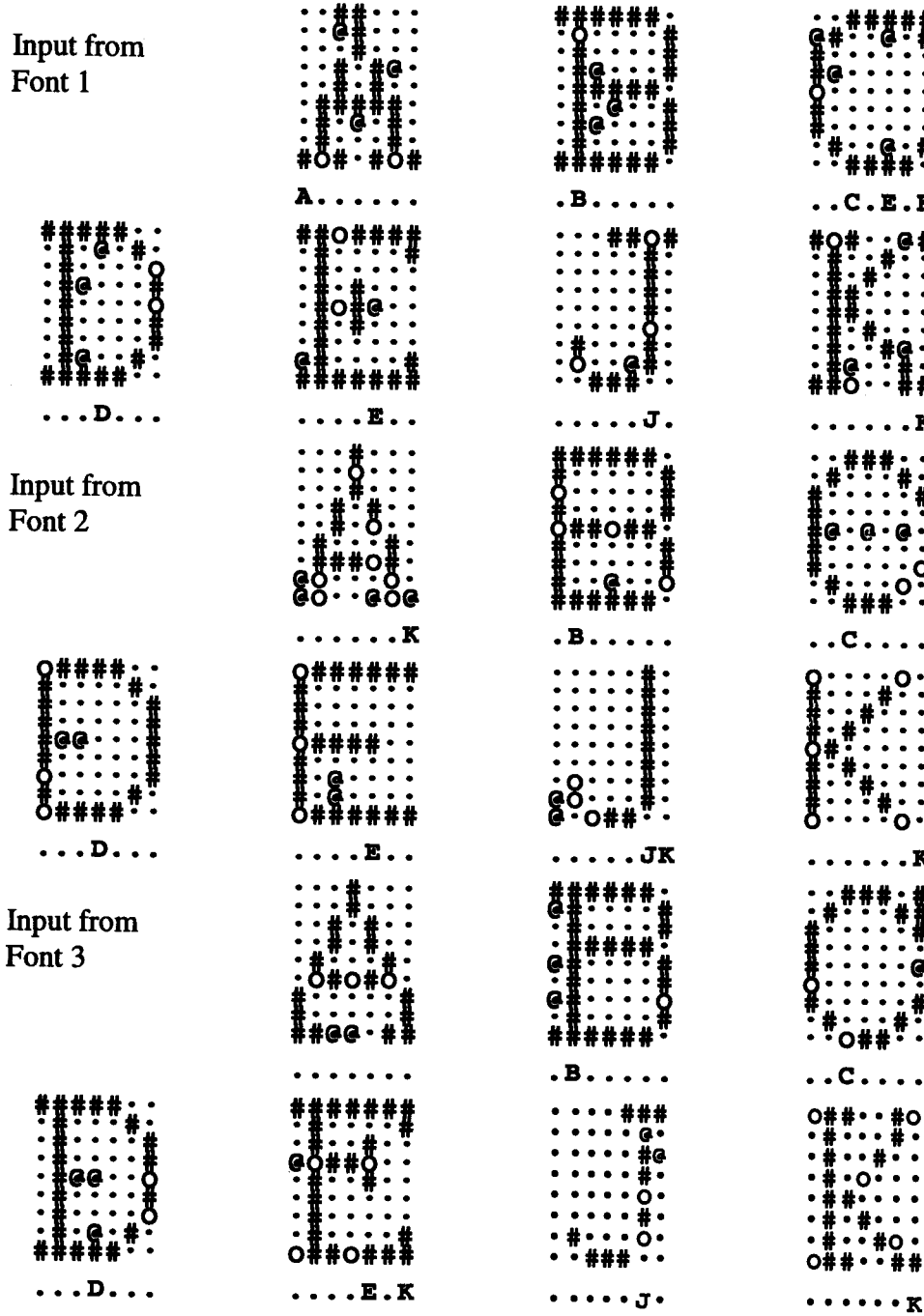

Figure 2.22 Classification of noisy input patterns using a perceptron.

patterns is a training input pattern with a few of its pixels changed. The pixels where the input pattern differs from the training pattern are indicated by @ for a pixel that is "on" now but was "off" in the training pattern, and O for a pixel that is "off" now but was originally "on."

2.3.4 Perceptron Learning Rule Convergence Theorem

The statement and proof of the perceptron learning rule convergence theorem given here are similar to those presented in several other sources [Hertz, Krogh, & Palmer, 1991; Minsky & Papert, 1988; Arbib, 1987]. Each of these provides a slightly different perspective and insights into the essential aspects of the rule. The fact that the weight vector is perpendicular to the plane separating the input patterns at each step of the learning processes [Hertz, Krogh, & Palmer, 1991] can be used to interpret the degree of difficulty of training a perceptron for different types of input.

The perceptron learning rule is as follows:

Given a finite set of P input training vectors

$$\mathbf{x}(p), \quad p = 1, \ldots, P,$$

each with an associated target value

$$t(p), \quad p = 1, \ldots, P,$$

which is either $+1$ or -1, and an activation function $y = f(y\_in)$, where

$$y = \begin{cases} 1 & \text{if } y\_in > \theta \\ 0 & \text{if } -\theta \leq y\_in \leq \theta \\ -1 & \text{if } y\_in < -\theta, \end{cases}$$

the weights are updated as follows:

If $y \neq t$, then

$$\mathbf{w}\ (\text{new}) = \mathbf{w}\ (\text{old}) + t\mathbf{x};$$

else

no change in the weights.

The perceptron learning rule convergence theorem is:

If there is a weight vector $\mathbf{w}^*$ such that $f(\mathbf{x}(p) \cdot \mathbf{w}^*) = t(p)$ for all p, then for any starting vector $\mathbf{w}$, the perceptron learning rule will converge to a weight vector (not necessarily unique and not necessarily $\mathbf{w}^*$) that gives the correct response for all training patterns, and it will do so in a finite number of steps.

The proof of the theorem is simplified by the observation that the training set can be considered to consist of two parts:

$$F^+ = \{\mathbf{x} \text{ such that the target value is } +1\}$$

and

$$F^- = \{\mathbf{x} \text{ such that the target value is } -1\}.$$

A new training set is then defined as

$$F = F^+ \cup -F^-,$$

where

$$-F^- = \{-\mathbf{x} \text{ such that } \mathbf{x} \text{ is in } F^-\}.$$

In order to simplify the algebra slightly, we shall assume, without loss of generality, that $\theta = 0$ and $\alpha = 1$ in the proof. The existence of a solution of the original problem, namely the existence of a weight vector $\mathbf{w}^*$ for which

$$\mathbf{x}\cdot\mathbf{w}^* > 0 \qquad \text{if } \mathbf{x} \text{ is in } F^+$$

and

$$\mathbf{x}\cdot\mathbf{w}^* < 0 \qquad \text{if } \mathbf{x} \text{ is in } F^-,$$

is equivalent to the existence of a weight vector $\mathbf{w}^*$ for which

$$\mathbf{x}\cdot\mathbf{w}^* > 0 \qquad \text{if } \mathbf{x} \text{ is in } F.$$

All target values for the modified training set are $+1$. If the response of the net is incorrect for a given training input, the weights are updated according to

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \mathbf{x}.$$

Note that the input training vectors must each have an additional component (which is always 1) included to account for the signal to the bias weight.

We now sketch the proof of this remarkable convergence theorem, because of the light that it sheds on the wide variety of forms of perceptron learning that are guaranteed to converge. As mentioned, we assume that the training set has been modified so that all targets are $+1$. Note that this will involve reversing the sign of all components (including the input component corresponding to the bias) for any input vectors for which the target was originally -1.

We now consider the sequence of input training vectors for which a weight change occurs. We must show that this sequence is finite.

Let the starting weights be denoted by $\mathbf{w}(0)$, the first new weights by $\mathbf{w}(1)$, etc. If $\mathbf{x}(0)$ is the first training vector for which an error has occurred, then

$$\mathbf{w}(1) = \mathbf{w}(0) + \mathbf{x}(0) \qquad (\text{where, by assumption, } \mathbf{x}(0)\cdot\mathbf{w}(0) \leq 0).$$

If another error occurs, we denote the vector $\mathbf{x}(1)$; $\mathbf{x}(1)$ may be the same as $\mathbf{x}(0)$ if no errors have occurred for any other training vectors, or $\mathbf{x}(1)$ may be different from $\mathbf{x}(0)$. In either case,

$$\mathbf{w}(2) = \mathbf{w}(1) + \mathbf{x}(1) \qquad (\text{where, by assumption, } \mathbf{x}(1)\cdot\mathbf{w}(1) \leq 0).$$

At any stage, say, k, of the process, the weights are changed if and only if the current weights fail to produce the correct (positive) response for the current input vector, i.e., if $\mathbf{x}(k-1)\cdot\mathbf{w}(k-1) \leq 0$. Combining the successive weight changes gives

$$\mathbf{w}(k) = \mathbf{w}(0) + \mathbf{x}(0) + \mathbf{x}(1) + \mathbf{x}(2) + \cdots + \mathbf{x}(k-1).$$

We now show that k cannot be arbitrarily large.

Let $\mathbf{w}^*$ be a weight vector such that $\mathbf{x}\cdot\mathbf{w}^* > 0$ for all training vectors in F. Let $m = \min\{\mathbf{x}\cdot\mathbf{w}^*\}$, where the minimum is taken over all training vectors in F; this minimum exists as long as there are only finitely many training vectors. Now,

$$\begin{aligned}\mathbf{w}(k)\cdot\mathbf{w}^* &= [\mathbf{w}(0) + \mathbf{x}(0) + \mathbf{x}(1) + \mathbf{x}(2) + \cdots + \mathbf{x}(k-1)]\cdot\mathbf{w}^* \\ &\geq \mathbf{w}(0)\cdot\mathbf{w}^* + km\end{aligned}$$

since $\mathbf{x}(i)\cdot\mathbf{w}^* \geq m$ for each i, $1 \leq i \leq P$.

The Cauchy-Schwartz inequality states that for any vectors $\mathbf{a}$ and $\mathbf{b}$,

$$(\mathbf{a}\cdot\mathbf{b})^2 \leq \|\mathbf{a}\|^2 \|\mathbf{b}\|^2,$$

or

$$\|\mathbf{a}\|^2 \geq \frac{(\mathbf{a}\cdot\mathbf{b})^2}{\|\mathbf{b}\|^2} \qquad (\text{for } \|\mathbf{b}\|^2 \neq 0).$$

Therefore,

$$\begin{aligned}\|\mathbf{w}(k)\|^2 &\geq \frac{(\mathbf{w}(k)\cdot\mathbf{w}^*)^2}{\|\mathbf{w}^*\|^2} \\ &\geq \frac{(\mathbf{w}(0)\cdot\mathbf{w}^* + km)^2}{\|\mathbf{w}^*\|^2}.\end{aligned}$$

This shows that the squared length of the weight vector grows faster than k^2, where k is the number of time the weights have changed.

However, to show that the length cannot continue to grow indefinitely, consider

$$\mathbf{w}(k) = \mathbf{w}(k-1) + \mathbf{x}(k-1),$$

together with the fact that

$$\mathbf{x}(k-1)\cdot\mathbf{w}(k-1) \leq 0.$$

By simple algebra,

$$\begin{aligned}\|\mathbf{w}(k)\|^2 &= \|\mathbf{w}(k-1)\|^2 + 2\mathbf{x}(k-1)\cdot\mathbf{w}(k-1) + \|\mathbf{x}(k-1)\|^2 \\ &\leq \|\mathbf{w}(k-1)\|^2 + \|\mathbf{x}(k-1)\|^2.\end{aligned}$$

Now let $M = \max \{\| \mathbf{x} \|^2 \text{ for all } \mathbf{x} \text{ in the training set}\}$; then

$$\begin{aligned}\|\mathbf{w}(k)\|^2 &\leq \|\mathbf{w}(k-1)\|^2 + \|\mathbf{x}(k-1)\|^2 \\ &\leq \|\mathbf{w}(k-2)\|^2 + \|\mathbf{x}(k-2)\|^2 + \|\mathbf{x}(k-1)\|^2 \\ &\vdots \\ &\leq \|\mathbf{w}(0)\|^2 + \|\mathbf{x}(0)\|^2 + \cdots + \|\mathbf{x}(k-1)\|^2 \\ &\leq \|\mathbf{w}(0)\|^2 + kM.\end{aligned}$$

Thus, the squared length grows less rapidly than linearly in k.

Combining the inequalities

$$\|\mathbf{w}(k)\|^2 \geq \frac{(\mathbf{w}(0)\mathbf{w}^* + km)^2}{\|\mathbf{w}^*\|^2}$$

and

$$\|\mathbf{w}(k)\|^2 \leq \|\mathbf{w}(0)\|^2 + kM$$

shows that the number of times that the weights may change is bounded. Specifically,

$$\frac{(\mathbf{w}(0)\cdot\mathbf{w}^* + km)^2}{\|\mathbf{w}^*\|^2} \leq \|\mathbf{w}(k)\|^2 \leq \|\mathbf{w}(0)\|^2 + kM.$$

Again, to simplify the algebra, assume (without loss of generality) that $\mathbf{w}(0) = 0$. Then the maximum possible number of times the weights may change is given by

$$\frac{(km)^2}{\|\mathbf{w}^*\|^2} \leq kM,$$

or

$$k \leq \frac{M\,\|\mathbf{w}^*\|^2}{m^2}.$$

Since the assumption that $\mathbf{w}^*$ exists can be restated, without loss of generality, as the assumption that there is a solution weight vector of unit length (and the definition of m is modified accordingly), the maximum number of weight updates is M/m^2. Note, however, that many more computations may be required, since very few input vectors may generate an error during any one epoch of training. Also, since $\mathbf{w}^*$ is unknown (and therefore, so is m), the number of weight updates cannot be predicted from the preceding inequality.

The foregoing proof shows that many variations in the perceptron learning rule are possible. Several of these variations are explicitly mentioned in Chapter 11 of Minsky and Papert (1988).

The original restriction that the coefficients of the patterns be binary is un-

necessary. All that is required is that there be a finite maximum norm of the training vectors (or at least a finite upper bound to the norm). Training may take a long time (a large number of steps) if there are training vectors that are very small in norm, since this would cause small m to have a small value. The argument of the proof is unchanged if a nonzero value of θ is used (although changing the value of θ may change a problem from solvable to unsolvable or vice versa). Also, the use of a learning rate other than 1 will not change the basic argument of the proof (see Exercise 2.8). Note that there is no requirement that there can be only finitely many training vectors, as long as the norm of the training vectors is bounded (and bounded away from 0 as well). The actual target values do not matter, either; the learning law simply requires that the weights be incremented by the input vector (or a multiple of it) whenever the response of the net is incorrect (and that the training vectors can be stated in such a way that they all should give the same response of the net).

Variations on the learning step include setting the learning rate α to any nonnegative constant (Minsky starts by setting it specifically to 1), setting α to $1/\|\mathbf{x}\|$ so that the weight change is a unit vector, and setting α to $(\mathbf{x}\cdot\mathbf{w})/\|\mathbf{x}\|^2$ (which makes the weight change just enough for the pattern $\mathbf{x}$ to be classified correctly at this step).

Minsky sets the initial weights equal to an arbitrary training pattern. Others usually indicate small random values.

Note also that since the procedure will converge from an arbitrary starting set of weights, the process is error correcting, as long as the errors do not occur too often (and the process is not stopped before error correction occurs).

2.4 ADALINE

The ADALINE (ADaptive Linear NEuron) [Widrow & Hoff, 1960] typically uses bipolar (1 or -1) activations for its input signals and its target output (although it is not restricted to such values). The weights on the connections from the input units to the ADALINE are adjustable. The ADALINE also has a bias, which acts like an adjustable weight on a connection from a unit whose activation is always 1.

In general, an ADALINE can be trained using the delta rule, also known as the least mean squares (LMS) or Widrow-Hoff rule. The rule (Section 2.4.2) can also be used for single-layer nets with several output units; an ADALINE is a special case in which there is only one output unit. During training, the activation of the unit is its net input, i.e., the activation function is the identity function. The learning rule minimizes the mean squared error between the activation and the target value. This allows the net to continue learning on all training patterns, even after the correct output value is generated (if a threshold function is applied) for some patterns.

After training, if the net is being used for pattern classification in which the desired output is either a $+1$ or a -1, a threshold function is applied to the net

input to obtain the activation. If the net input to the ADALINE is greater than or equal to 0, then its activation is set to 1; otherwise it is set to −1. Any problem for which the input patterns corresponding to the output value +1 are linearly separable from input patterns corresponding to the output value −1 can be modeled successfully by an ADALINE unit. An application algorithm is given in Section 2.4.3 to illustrate the use of the activation function after the net is trained.

In Section 2.4.4, we shall see how a heuristic learning rule can be used to train a multilayer combination of ADALINES, known as a MADALINE.

2.4.1 Architecture

An ADALINE is a single unit (neuron) that receives input from several units. It also receives input from a "unit" whose signal is always +1, in order for the bias weight to be trained by the same process (the delta rule) as is used to train the other weights. A single ADALINE is shown in Figure 2.23.

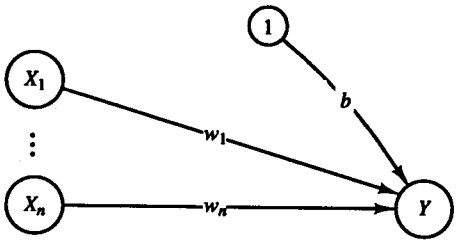

Figure 2.23 Architecture of an ADALINE.

Several ADALINES that receive signals from the same input units can be combined in a single-layer net, as described for the perceptron (Section 2.3.3). If, however, ADALINES are combined so that the output from some of them becomes input for others of them, then the net becomes multilayer, and determining the weights is more difficult. Such a multilayer net, known as a MADALINE, is considered in Section 2.4.5.

2.4.2 Algorithm

A training algorithm for an ADALINE is as follows:

Step 0. Initialize weights.
(Small random values are usually used.)
Set learning rate α.
(See comments following algorithm.)

Step 1. While stopping condition is false, do Steps 2–6.

Step 2. For each bipolar training pair s:t, do Steps 3–5.

Step 3. Set activations of input units, $i = 1, \ldots, n$:

$$x_i = s_i.$$

Step 4. Compute net input to output unit:

$$y\_in = b + \sum_i x_i w_i.$$

Step 5. Update bias and weights, $i = 1, \ldots, n$:

$$b(\text{new}) = b(\text{old}) + \alpha(t - y\_in).$$

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y\_in)x_i.$$

Step 6. Test for stopping condition:
If the largest weight change that occurred in Step 2 is smaller than a specified tolerance, then stop; otherwise continue.

Setting the learning rate to a suitable value requires some care. According to Hecht-Nielsen (1990), an upper bound for its value can be found from the largest eigenvalue of the correlation matrix R of the input (row) vectors $\mathbf{x}(p)$:

$$R = \frac{1}{P} \sum_{p=1}^{P} \mathbf{x}(p)^T \mathbf{x}(p),$$

namely,

$$\alpha < \text{one-half the largest eigenvalue of } R.$$

However, since R does not need to be calculated to compute the weight updates, it is common simply to take a small value for α (such as $\alpha = .1$) initially. If too large a value is chosen, the learning process will not converge; if too small a value is chosen, learning will be extremely slow [Hecht-Nielsen, 1990]. The choice of learning rate and methods of modifying it are considered further in Chapter 6. For a single neuron, a practical range for the learning rate α is $0.1 \leq n\alpha \leq 1.0$, where n is the number of input units [Widrow, Winter & Baxter, 1988].

The proof of the convergence of the ADALINE training process is essentially contained in the derivation of the delta rule, which is given in Section 2.4.4.

2.4.3 Applications

After training, an ADALINE unit can be used to classify input patterns. If the target values are bivalent (binary or bipolar), a step function can be applied as the

activation function for the output unit. The following procedure shows the step function for bipolar targets, the most common case:

Step 0. Initialize weights
(from ADALINE training algorithm given in Section 2.4.2).

Step 1. For each bipolar input vector **x**, do Steps 2–4.

Step 2. Set activations of the input units to **x**.

Step 3. Compute net input to output unit:

$$y\_in = b + \sum_i x_i w_i.$$

Step 4. Apply the activation function:

$$y = \begin{cases} 1 & \text{if } y\_in \geq 0; \\ -1 & \text{if } y\_in < 0. \end{cases}$$

Simple examples

The weights (and biases) in Examples 2.16–2.19 give the minimum total squared error for each set of training patterns. Good approximations to these values can be found using the algorithm in Section 2.4.2 with a small learning rate.

Example 2.16 An ADALINE for the AND function: binary inputs, bipolar targets

Even though the ADALINE was presented originally for bipolar inputs and targets, the delta rule also applies to binary input. In this example, we consider the AND function with binary input and bipolar targets. The function is defined by the following four training patterns:

x_1	x_2	t
1	1	1
1	0	−1
0	1	−1
0	0	−1

As indicated in the derivation of the delta rule (Section 2.4.4), an ADALINE is designed to find weights that minimize the total error

$$E = \sum_{p=1}^{4} (x_1(p)w_1 + x_2(p)w_2 + w_0 - t(p))^2,$$

where

$$x_1(p)w_1 + x_2(p)w_2 + w_0$$

is the net input to the output unit for pattern p and $t(p)$ is the associated target for pattern p.

Weights that minimize this error are

$$w_1 = 1$$

and

$$w_2 = 1,$$

with the bias

$$w_0 = -\frac{3}{2}.$$

Thus, the separating line is

$$x_1 + x_2 - \frac{3}{2} = 0.$$

The total squared error for the four training patterns with these weights is 1.

A minor modification to Example 2.11 (setting $\theta = 0$) shows that for the perceptron, the boundary line is

$$x_2 = -\frac{2}{3}x_1 + \frac{4}{3}.$$

(The two boundary lines coincide when $\theta = 0$.) The total squared error for the minimizing weights found by the perceptron is 10/9.

Example 2.17 An ADALINE for the AND function: bipolar inputs and targets

The weights that minimize the total error for the bipolar form of the AND function are

$$w_1 = \frac{1}{2}$$

and

$$w_2 = \frac{1}{2},$$

with the bias

$$w_0 = -\frac{1}{2}.$$

Thus, the separating line is

$$\frac{1}{2}x_1 + \frac{1}{2}x_2 - \frac{1}{2} = 0,$$

which is of course the same line as

$$x_1 + x_2 - 1 = 0,$$

as found by the perceptron in Example 2.12.

Example 2.18 An ADALINE for the AND NOT function: bipolar inputs and targets

The logic function x_1 AND NOT x_2 is defined by the following bipolar input and target patterns:

x_1	x_2	t
1	1	−1
1	−1	1
−1	1	−1
−1	−1	−1

Weights that minimize the total squared error for the bipolar form of the AND NOT function are

$$w_1 = \frac{1}{2}$$

and

$$w_2 = -\frac{1}{2},$$

with the bias

$$w_0 = -\frac{1}{2}.$$

Thus, the separating line is

$$\frac{1}{2}x_1 - \frac{1}{2}x_2 - \frac{1}{2} = 0.$$

Example 2.19 An ADALINE for the OR function: bipolar inputs and targets

The logic function x_1 OR x_2 is defined by the following bipolar input and target patterns:

x_1	x_2	t
1	1	1
1	−1	1
−1	1	1
−1	−1	−1

Weights that minimize the total squared error for the bipolar form of the OR function are

$$w_1 = \frac{1}{2}$$

and

$$w_2 = \frac{1}{2},$$

with the bias

$$w_0 = \frac{1}{2}.$$

Thus, the separating line is

$$\frac{1}{2}x_1 + \frac{1}{2}x_2 + \frac{1}{2} = 0.$$

2.4.4 Derivations

Delta rule for single output unit

The delta rule changes the weights of the neural connections so as to minimize the difference between the net input to the output unit, *y_in*, and the target value *t*. The aim is to minimize the error over all training patterns. However, this is accomplished by reducing the error for each pattern, one at a time. Weight corrections can also be accumulated over a number of training patterns (so-called batch updating) if desired. In order to distinguish between the fixed (but arbitrary) index for the weight whose adjustment is being determined in the derivation that follows and the index of summation needed in the derivation, we use the index *I* for the weight and the index *i* for the summation. We shall return to the more standard lowercase indices for weights whenever this distinction is not needed. The delta rule for adjusting the *I*th weight (for each pattern) is

$$\Delta w_I = \alpha(t - y\_in)x_I.$$

The nomenclature we use in the derivation is as follows:

x vector of activations of input units, an *n*-tuple.

y_in the net input to output unit *Y* is

$$y\_in = \sum_{i=1}^{n} x_i w_i.$$

t target output.

Derivation. The squared error for a particular training pattern is

$$E = (t - y\_in)^2.$$

E is a function of all of the weights, w_i, $i = 1, \ldots, n$. The gradient of *E* is the vector consisting of the partial derivatives of *E* with respect to each of the weights. The gradient gives the direction of most rapid increase in *E*; the opposite direction

gives the most rapid decrease in the error. The error can be reduced by adjusting the weight w_I in the direction of $-\dfrac{\partial E}{\partial w_I}$.

Since $y\_in = \sum_{i=1}^{n} x_i w_i$,

$$\frac{\partial E}{\partial w_I} = -2(t - y\_in)\frac{\partial y\_in}{\partial w_I}$$

$$= -2(t - y\_in)x_I.$$

Thus, the local error will be reduced most rapidly (for a given learning rate) by adjusting the weights according to the delta rule,

$$\Delta w_I = \alpha(t - y\_in)x_I.$$

Delta rule for several output units

The derivation given in this subsection allows for more than one output unit. The weights are changed to reduce the difference between the net input to the output unit, y_in_J, and the target value t_J. This formulation reduces the error for each pattern. Weight corrections can also be accumulated over a number of training patterns (so-called batch updating) if desired.

The delta rule for adjusting the weight from the Ith input unit to the Jth output unit (for each pattern) is

$$\Delta w_{IJ} = \alpha(t_J - y\_in_J)x_I.$$

Derivation. The squared error for a particular training pattern is

$$E = \sum_{j=1}^{m} (t_j - y\_in_j)^2.$$

E is a function of all of the weights. The gradient of E is a vector consisting of the partial derivatives of E with respect to each of the weights. This vector gives the direction of most rapid increase in E; the opposite direction gives the direction of most rapid decrease in the error. The error can be reduced most rapidly by adjusting the weight w_{IJ} in the direction of $-\partial E/\partial w_{IJ}$.

We now find an explicit formula for $\partial E/\partial w_{IJ}$ for the arbitrary weight w_{IJ}. First, note that

$$\frac{\partial E}{\partial w_{IJ}} = \frac{\partial}{\partial w_{IJ}} \sum_{j=1}^{m} (t_j - y\_in_j)^2$$

$$= \frac{\partial}{\partial w_{IJ}} (t_J - y\_in_J)^2,$$

since the weight w_{IJ} influences the error only at output unit Y_J. Furthermore, using the fact that

$$y\_in_J = \sum_{i=1}^{n} x_i w_{iJ},$$

we obtain

$$\frac{\partial E}{\partial w_{IJ}} = -2(t_J - y\_in_J)\frac{\partial y\_in_J}{\partial w_{IJ}}$$
$$= -2(t_J - y\_in_J)x_I.$$

Thus, the local error will be reduced most rapidly (for a given learning rate) by adjusting the weights according to the delta rule,

$$\Delta w_{IJ} = \alpha(t_J - y\_in_J)x_I.$$

The preceding two derivations of the delta rule can be generalized to the case where the training data are only samples from a larger data set, or probability distribution. Minimizing the error for the training set will also minimize the expected value for the error of the underlying probability distribution. (See Widrow & Lehr, 1990 or Hecht-Nielsen, 1990 for a further discussion of the matter.)

2.4.5 MADALINE

As mentioned earlier, a MADALINE consists of Many ADAptive Linear NEurons arranged in a multilayer net. The examples given for the perceptron and the derivation of the delta rule for several output units both indicate there is essentially no change in the process of training if several ADALINE units are combined in a single-layer net. In this section we will discuss a MADALINE with one hidden layer (composed of two hidden ADALINE units) and one output ADALINE unit. Generalizations to more hidden units, more output units, and more hidden layers, are straightforward.

Architecture

A simple MADALINE net is illustrated in Figure 2.24. The outputs of the two hidden ADALINES, z_1 and z_2, are determined by signals from the same input units X_1 and X_2. As with the ADALINES discussed previously, each output signal is the result of applying a threshold function to the unit's net input. Thus, y is a nonlinear function of the input vector (x_1, x_2). The use of the hidden units, Z_1 and Z_2, give the net computational capabilities not found in single layer nets, but also complicate the training process. In the next section we consider two training algorithms for a MADALINE with one hidden layer.

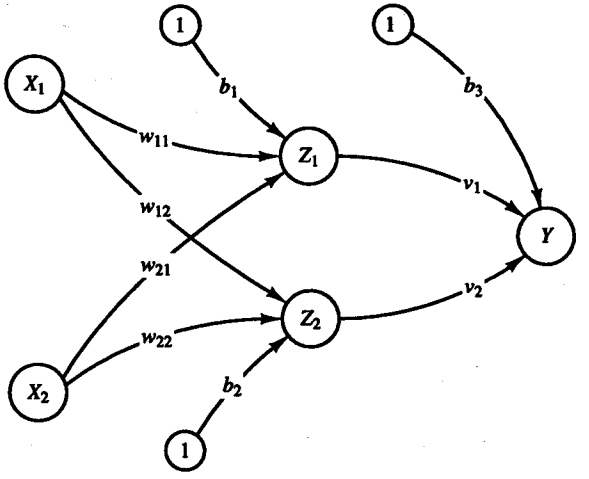

Figure 2.24 A MADALINE with two hidden ADALINES and one output ADALINE.

Algorithm

In the MRI algorithm (the original form of MADALINE training) [Widrow and Hoff, 1960], only the weights for the hidden ADALINES are adjusted; the weights for the output unit are fixed. The MRII algorithm [Widrow, Winter, & Baxter, 1987] provides a method for adjusting all weights in the net.

We consider first the MRI algorithm; the weights v_1 and v_2 and the bias b_3 that feed into the output unit Y are determined so that the response of unit Y is 1 if the signal it receives from either Z_1 or Z_2 (or both) is 1 and is -1 if both Z_1 and Z_2 send a signal of -1. In other words, the unit Y performs the logic function OR on the signals it receives from Z_1 and Z_2. The weights into Y are

$$v_1 = \frac{1}{2}$$

and

$$v_2 = \frac{1}{2},$$

with the bias

$$b_3 = \frac{1}{2}$$

(see Example 2.19). The weights on the first hidden ADALINE (w_{11} and w_{21}) and the weights on the second hidden ADALINE (w_{12} and w_{22}) are adjusted according to the algorithm.

Training Algorithm for MADALINE (MRI). The activation function for units Z_1, Z_2, and Y is

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0; \\ -1 & \text{if } x < 0. \end{cases}$$

Step 0. Initialize weights:
Weights v_1 and v_2 and the bias b_3 are set as described;
small random values are usually used for ADALINE weights.
Set the learning rate α as in the ADALINE training algorithm (a small value).

Step 1. While stopping condition is false, do Steps 2–8.

Step 2. For each bipolar training pair, s:t, do Steps 3–7.

Step 3. Set activations of input units:

$$x_i = s_i.$$

Step 4. Compute net input to each hidden ADALINE unit:

$$z\_in_1 = b_1 + x_1 w_{11} + x_2 w_{21},$$

$$z\_in_2 = b_2 + x_1 w_{12} + x_2 w_{22}.$$

Step 5. Determine output of each hidden ADALINE unit:

$$z_1 = f(z\_in_1),$$

$$z_2 = f(z\_in_2).$$

Step 6. Determine output of net:

$$y\_in = b_3 + z_1 v_1 + z_2 v_2;$$

$$y = f(y\_in).$$

Step 7. Determine error and update weights:
If $t = y$, no weight updates are performed.
Otherwise:
If $t = 1$, then update weights on Z_J,
the unit whose net input is closest to 0,

$$b_J(\text{new}) = b_J(\text{old}) + \alpha(1 - z\_in_J),$$

$$w_{iJ}(\text{new}) = w_{iJ}(\text{old}) + \alpha(1 - z\_in_J)x_i;$$

If $t = -1$, then update weights on all units Z_k that have positive net input,

$$b_k(\text{new}) = b_k(\text{old}) + \alpha(-1 - z\_in_k),$$

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha(-1 - z\_in_k)x_i.$$

Step 8. Test stopping condition.
If weight changes have stopped (or reached an acceptable level), or if a specified maximum number of weight update iterations (Step 2) have been performed, then stop; otherwise continue.

Step 7 is motivated by the desire to (1) update the weights only if an error occurred and (2) update the weights in such a way that it is more likely for the net to produce the desired response.

If $t = 1$ and error has occurred, it means that all Z units had value -1 and at least one Z unit needs to have a value of $+1$. Therefore, we take Z_J to be the Z unit whose net input is closest to 0 and adjust its weights (using ADALINE training with a target value of $+1$):

$$b_J(\text{new}) = b_J(\text{old}) + \alpha(1 - z\_in_J),$$

$$w_{iJ}(\text{new}) = w_{iJ}(\text{old}) + \alpha(1 - z\_in_J)x_i.$$

If $t = -1$ and error has occurred, it means that at least one Z unit had value $+1$ and all Z units must have value -1. Therefore, we adjust the weights on all of the Z units with positive net input, (using ADALINE training with a target of -1):

$$b_k(\text{new}) = b_k(\text{old}) + \alpha(-1 - z\_in_k),$$

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha(-1 - z\_in_k)x_i.$$

MADALINES can also be formed with the weights on the output unit set to perform some other logic function such as AND or, if there are more than two hidden units, the "majority rule" function. The weight update rules would be modified to reflect the logic function being used for the output unit [Widrow & Lehr, 1990].

A more recent MADALINE training rule, called MRII [Widrow, Winter, & Baxter, 1987], allows training for weights in all layers of the net. As in earlier MADALINE training, the aim is to cause the least disturbance to the net at any step of the learning process, in order to cause as little "unlearning" of patterns for which the net had been trained previously. This is sometimes called the "don't rock the boat" principle. Several output units may be used; the total error for any input pattern (used in Step 7b) is the sum of the squares of the errors at each output unit.

Training Algorithm for MADALINE (MRII).

Step 0. Initialize weights:
Set the learning rate α.

Step 1. While stopping condition is false, do Steps 2–8.

Step 2. For each bipolar training pair, s:t, do Steps 3–7.

Step 3–6. Compute output of net as in the MRI algorithm.

Step 7. Determine error and update weights if necessary:

If $t \neq y$, do Steps 7a–b for each hidden unit whose net input is sufficiently close to 0 (say, between $-.25$ and .25). Start with the unit whose net input is closest to 0, then for the next closest, etc.

Step 7a. Change the unit's ouput (from $+1$ to -1, or vice versa).

Step 7b. Recompute the response of the net.
If the error is reduced:
adjust the weights on this unit (use its newly assigned output value as target and apply the Delta Rule).

Step 8. Test stopping condition.
If weight changes have stopped (or reached an acceptable level), or if a specified maximum number of weight update iterations (Step 2) have been performed, then stop; otherwise continue.

A further modification is the possibility of attempting to modify pairs of units at the first layer after all of the individual modifications have been attempted. Similarly adaptation could then be attempted for triplets of units.

Application

Example 2.20 Training a MADALINE for the XOR function

This example illustrates the use of the MRI algorithm to train a MADALINE to solve the XOR problem. Only the computations for the first weight updates are shown.

The training patterns are:

x_1	x_2	t
1	1	−1
1	−1	1
−1	1	1
−1	−1	−1

Step 0.

The weights into Z_1 and into Z_2 are small random values; the weights into Y are those found in Example 2.19. The learning rate, α, is .5.

Weights into Z_1			Weights into Z_2			Weights into Y		
w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2	v_1	v_2	b_3
.05	.2	.3	.1	.2	.15	.5	.5	.5

Step 1. Begin training.

Step 2. For the first training pair, (1, 1): −1

Step 3. $x_1 = 1, \quad x_2 = 1$

Step 4. $z\_in_1 = .3 + .05 + .2 = .55,$
$z\_in_2 = .15 + .1 + .2 = .45.$

Step 5. $z_1 = 1$,
$z_2 = 1$.
Step 6. $y\_in = .5 + .5 + .5$;
$y = 1$.
Step 7. $t - y = -1 - 1 = -2 \neq 0$, so an error occurred.
Since $t = -1$, and both Z units have positive net input,
update the weights on unit Z_1 as follows:

$$b_1(\text{new}) = b_1(\text{old}) + \alpha(-1 - z\_in_1)$$
$$= .3 + (.5)(-1.55)$$
$$= -.475$$
$$w_{11}(\text{new}) = w_{11}(\text{old}) + \alpha(-1 - z\_in_1)x_1$$
$$= .05 + (.5)(-1.55)$$
$$= -.725$$
$$w_{21}(\text{new}) = w_{21}(\text{old}) + \alpha(-1 - z\_in_1)x_2$$
$$= .2 + (.5)(-1.55)$$
$$= -.575$$

update the weights on unit Z_2 as follows:

$$b_2(\text{new}) = b_2(\text{old}) + \alpha(-1 - z\_in_2)$$
$$= .15 + (.5)(-1.45)$$
$$= -.575$$
$$w_{12}(\text{new}) = w_{12}(\text{old}) + \alpha(-1 - z\_in_2)x_1$$
$$= .1 + (.5)(-1.45)$$
$$= -.625$$
$$w_{22}(\text{new}) = w_{22}(\text{old}) + \alpha(-1 - z\_in_2)x_2$$
$$= .2 + (.5)(-1.45)$$
$$= -.525$$

After four epochs of training, the final weights are found to be:

$$w_{11} = -0.73 \qquad w_{12} = 1.27$$
$$w_{21} = 1.53 \qquad w_{22} = -1.33$$
$$b_1 = -0.99 \qquad b_2 = -1.09$$

Example 2.21 Geometric interpretation of MADALINE weights

The positive response region for the Madaline trained in the previous example is the union of the regions where each of the hidden units have a positive response. The

decision boundary for each hidden unit can be calculated as described in Section 2.1.3.

For hidden unit Z_1, the boundary line is

$$x_2 = -\frac{w_{11}}{w_{21}} x_1 - \frac{b_1}{w_{21}}$$

$$= \frac{0.73}{1.53} x_1 + \frac{0.99}{1.53}$$

$$= 0.48\, x_1 + 0.65$$

For hidden unit Z_2, the boundary line is

$$x_2 = -\frac{w_{12}}{w_{22}} x_1 - \frac{b_2}{w_{22}}$$

$$= \frac{1.27}{1.33} x_1 + \frac{1.09}{1.33}$$

$$= 0.96\, x_1 - 0.82$$

These regions are shown in Figures 2.25 and 2.26. The response diagram for the MADALINE is illustrated in Figure 2.27.

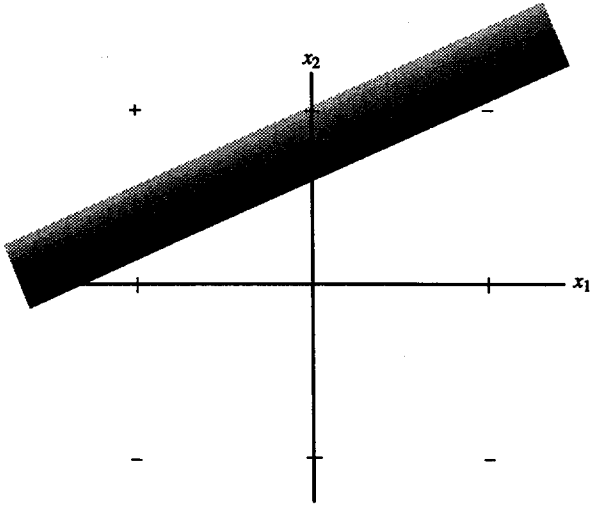

Figure 2.25 Positive response region for Z_1.

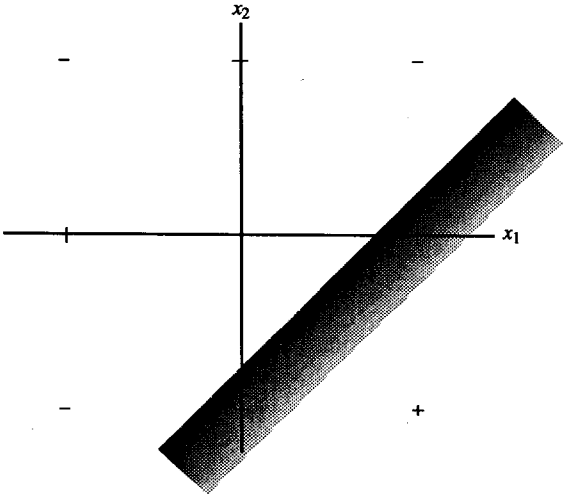

Figure 2.26 Positive response region for Z_2.

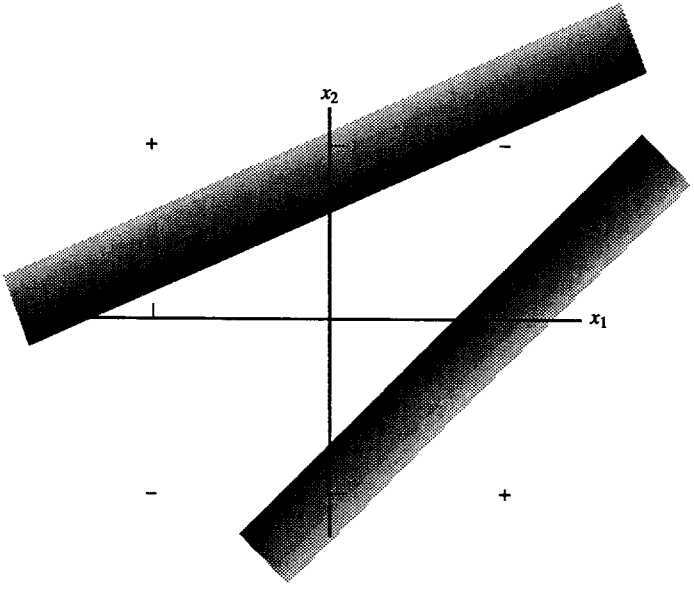

Figure 2.27 Positive response region for MADALINE for XOR function.

Discussion

The construction of sample multilayer nets may provide insights into the appropriate choice of parameters for multilayer nets in general, such as those trained using backpropagation (discussed in Chapter 6). For example, if the input patterns fall into regions that can be bounded (approximately) by a number of lines or planes, then the number of hidden units can be estimated.

It is possible to construct a net with $2p$ hidden units (in a single layer) that will learn p bipolar input training patterns (each with an associated bipolar target value) perfectly. Of course, that is not the primary (or at least not the exclusive) goal of neural nets; generalization is also important and will not be particularly good with so many hidden units. In addition, the training time and the number of interconnections will be unnecessarily large. However, $2p$ certainly gives an upper bound on the number of hidden units we might consider using.

For input that is to be assigned to different categories (the kind of input we have been considering in this chapter), we see that the regions which each neuron separates are bounded by straight lines. Closed regions (convex polygons) can be bounded by taking the intersection of several half-planes (bounded by the separating lines described earlier). Thus a net with one hidden layer (with p units) can learn a response region bounded by p straight lines. If responses in the same category occur in more than one disjoint region of the input space, an additional hidden layer to combine these regions will make training easier.

2.5 SUGGESTIONS FOR FURTHER STUDY

2.5.1 Readings

Hebb rule

The description of the original form of the Hebb rule is found in

HEBB, D. O. (1949). *The Organization of Behavior*. New York: John Wiley & Sons. Introduction and Chapter 4 reprinted in Anderson and Rosenfeld (1988), pp. 45–56.

Perceptrons

The description of the perceptron, as presented in this chapter, is based on

BLOCK, H. D. (1962). "The Perceptron: A Model for Brain Functioning, I." *Reviews of Modern Physics*, 34:123–135. Reprinted in Anderson and Rosenfeld (1988), pp. 138–150.

There are many types of perceptrons; for more complete coverage, see:

MINSKY, M. L., & S. A. PAPERT. (1988). *Perceptrons, Expanded Edition*. Cambridge, MA: MIT Press. Original Edition, 1969.

ROSENBLATT, F. (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." *Psychological Review*, 65:386–408. Reprinted in Anderson and Rosenfeld (1988), pp. 92–114.

ROSENBLATT, F. (1962). *Principles of Neurodynamics*. New York: Spartan.

ADALINE and MADALINE

For further discussion of ADALINE and MADALINE, see

WIDROW, B., & M. E. HOFF, JR. (1960). "Adaptive Switching Circuits." *IRE WESCON Convention Record*, part 4, pp. 96–104. Reprinted in Anderson and Rosenfeld (1988), pp. 126–134.

WIDROW, B., and S. D. STEARNS. (1985). *Adaptive Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall.

WIDROW, B. & M. A. LEHR. (1990). "30 Years of Adaptive Neural Networks: Perceptron, MADALINE, and Backpropagation," Proceeding of the IEEE, 78(9):1415–1442.